

DEEPLOGIX

Capstone Project Report End-Semester Evaluation

Submitted by:

Harsh Lakyan (102203215)

Aditya Tejpal (102203330)

Pankaj Sheokand (102203410)

Arpit Aggarwal (102203872)

BE Fourth Year-BE COE

CPG No:142

Under the Mentorship of

Dr. Sunita Gharwal

Associate Professor



Computer Science and Engineering Department

Thapar Institute of Engineering and Technology

December (2025)

ABSTRACT

DeepLogix is an intelligent voice logging and personal productivity analysis system designed to capture, process, and interpret spoken conversations in a structured and meaningful manner. The system integrates lightweight IoT hardware with a software-centric processing pipeline to transform raw voice input into actionable insights.

An ESP32-S3 microcontroller paired with an INMP441 I2S microphone captures audio and stores it locally in WAV format before transmitting it over Wi-Fi to a backend server. The backend, implemented using Flask, performs speech-to-text conversion and manages data handling and synchronization. Transcribed text is stored chronologically in a cloud database, enabling real-time access through a mobile application.

The mobile application provides users with conversation transcripts, daily summaries, keyword trends, and heuristic-based productivity indicators. Privacy is emphasized through the use of local AI models for conversational interaction and summarization, ensuring that sensitive user data is not shared with third-party cloud AI services.

By combining IoT-based voice acquisition, backend speech processing, real-time synchronization, and privacy-aware AI analysis, DeepLogix reduces reliance on manual note-taking, improves recall of spoken information, and supports reflective analysis of communication patterns. The system is designed as a scalable prototype, providing a foundation for future enhancements such as multilingual support, improved linguistic analytics, and on-device speech processing.

DECLARATION

We hereby declare that the design principles and working prototype model of the project entitled “DeepLogix” is an authentic record of our own work carried out in the Computer Science and Engineering Department, TIET, Patiala, under the guidance of Dr. Sunita Gharwal during 6th and 7th semester (2025).

Date: 23rd December 2025

Roll No.	Name	Signature
102203215	Harsh Lakyan	
102203330	Aditya Tejpal	
102203410	Pankaj Sheokhand	
102203872	Arpit Aggarwal	

Counter Signed By:

Mentor:

Dr. Sunita Gharwal

Associate Professor

Computer Science & Engineering Department

TIET, Patiala

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our mentor, **Dr. Sunita Garhwal**, for her invaluable guidance, continuous support, and encouragement throughout the course of this project. Her technical expertise and insightful suggestions have been indispensable in shaping our work and ensuring its successful completion.

We are also thankful to **Dr. Neeraj Kumar**, Head, Computer Science and Engineering Department, the entire faculty and staff of the Computer Science and Engineering Department, and our friends who devoted their valuable time and assisted us in every possible way towards the success of this project. We extend our heartfelt thanks to all those who contributed either directly or indirectly to the completion of this work.

Lastly, we would like to thank our families for their constant love, motivation, and encouragement. Their unwavering support and sacrifices have always inspired us to strive for the best, and we deeply admire their determination and care.

Date: 23rd December 2025

Roll No.	Name	Signature
102203215	Harsh Lakyan	
102203330	Aditya Tejpal	
102203410	Pankaj Sheokhand	
102203872	Arpit Aggarwal	

TABLE OF CONTENTS

ABSTRACT.....	i
DECLARATION.....	ii
ACKNOWLEDGEMENT.....	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ABBREVIATIONS	vi
CHAPTER.....	Page No.

1- INTRODUCTION	1
------------------------	----------

- 1.1 Project Overview
 - 1.1.1 Technical terminology
 - 1.1.2 Problem statement
 - 1.1.3 Goal
 - 1.1.4 Solution
- 1.2 Need Analysis
- 1.3 Research Gaps
- 1.4 Problem Definition and Scope
- 1.5 Assumptions and Constraints
- 1.6 Standards
- 1.7 Objectives
- 1.8 Methodology Used
- 1.9 Project Outcomes and Deliverables
- 1.10 Novelty of Work

2 - REQUIREMENT ANALYSIS	13
---------------------------------	-----------

- 2.1 Literature Survey

2.1.1 Related Work

2.1.2 Research Gaps of Existing Literature

2.1.3 Detailed Problem Analysis

2.1.4 Survey of Tools and Technologies Used

2.1.5 Summary

2.3 Software Requirements Specification

2.3.1 Introduction

2.3.1.1 Purpose

2.3.1.2 Intended Audience and Reading Suggestions

2.3.1.3 Project Scope

2.3.1.4 Overall Description

2.3.1.5 Product Perspective

2.3.1.6 Product Features

2.3.2 External Interface Requirements

2.3.2.1 User Interfaces

2.3.2.2 Hardware Interfaces

2.3.2.3 Software Interfaces

2.3.3 Other Non-functional Requirements

2.3.3.1 Performance Requirements

2.3.3.2 Safety Requirements

2.3.3.3 Security Requirements

2.4 Cost Analysis

2.5 Risk Analysis

3 -METHODOLOGY ADOPTED

28

3.1 Investigative Techniques

3.2 Proposed Solution

3.3 Work Breakdown Structure

3.4 Tools and Technologies Used

4 -DESIGN SPECIFICATIONS	33
4.1 System Architecture	
4.2 Design Level Diagrams	
4.3 .0	
4.4 User Interface Diagrams	
5 -IMPLEMENTATION AND EXPERIMENTAL RESULTS	43
5.1 Experimental Setup (or simulation)	
5.2 Experimental Analysis	
5.2.1 Data	
5.2.2 Performance Parameters	
5.3 Working of the project	
5.3.1 Procedural Workflow	
5.3.2 Algorithmic Approaches Used	
5.3.3 Project Deployment	
5.3.4 System Screenshots	
5.3.5 TestingProcess	
5.3.6 Test Plan	
5.3.7 Features to be tested	
5.3.8 Test Strategy	
5.3.9 Test Techniques	
5.3.10 Test Cases	
5.3.11 Test Results	
5.4 Results and Discussions	
5.5 Inferences Drawn	
5.6 Validation of Objectives	
6 CONCLUSIONS AND FUTURE DIRECTIONS	69
6.1 Conclusions	
6.2 Environmental, Economic and Societal Benefits	

6.3 Reflections

6.4 Future Work

7 -PROJECT METRICS **71**

7.1 Challenges Faced

7.2 Relevant Subjects

7.3 Interdisciplinary Knowledge Sharing

7.4 Peer Assessment Matrix

7.5 Role Playing and Work Schedule

7.6 Student Outcomes Description and Performance Indicators

7.7 Brief Analytical Assessment

APPENDIX A:

REFERENCES **77**

APPENDIX B:

PLAGIARISM REPORT **79**

LIST OF TABLES

Table No.	Caption	Page No.
Table 2.1.1	Research Findings	16-18
Table 2.3.1	Bills of material	24
Table 2.3.2	Software and Cloud Services	25
Table 2.3.3	Development Cost.	25-26
Table 2.3.4	Total Cost	26
Table 3.1.1	Investigative Technique	28-29
Table 5.4.1	Software Test Case Table	63-64
Table 5.4.2	Hardware Test Case Table	65
Table 5.4.3	Test Result	66
Table 7.4.1	Peer Assessment	72
Table 7.6.1	SO-SO7 Mapping for Course UCS794	74-76

LIST OF FIGURES

Figure No.	Caption	Page
4.1.1	Block Diagram	33
4.2.1	Use Case Diagram	35
4.2.2	Activity Diagram	37
4.2.3	Swimlane Diagram	38
4.2.4	DFD Level 0 Diagram	39
4.2.5	DFD Level 1 Diagram	40
4.2.6	DFD Level 2 Diagram	41
4.3.1	User Interface Diagram	42
5.3.1	Workflow Diagram	47
5.3.2	Deployment Diagram	50
5.3.3	Dashboard Screenshot	51
5.3.4	Keywords Detection	52
5.3.5	Settings	53
5.3.6	Competition	54
5.3.7	Summary	55
5.3.8	ChatBot	56
5.3.9	Transcription	57
5.3.10	Sign Up Page	58
5.3.11	Password Reset	59

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
DL	Deep Learning
ASR	Automatic Speech Recognition
IoT	Internet of Things
ESP32-S3	ESP32 System-on-Chip (S3 variant)
API	Application Programming Interface
REST API	Representational State Transfer API
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
Wi-Fi	Wireless Fidelity
SD Card	Secure Digital Card
I2S	Inter-Integrated Sound
SPI	Serial Peripheral Interface
WAV	Waveform Audio Format
CPU	Central Processing Unit
RAM	Random Access Memory
PD Score	Personal Development Score
LM	Language Model
PLM	Pre-trained Language Model
BERT	Bidirectional Encoder Representations from Transformers
RoBERTa	Robustly Optimized BERT Pretraining Approach

DeBERTa	Decoding-enhanced BERT with disentangled attention
GPT	Generative Pre-trained Transformer
T5	Text-To-Text Transfer Transformer
BART	Bidirectional and Auto-Regressive Transformers
PEGASUS	Pre-Training with Extracted Gap-Sentences for Abstractive Summarization
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
BLEU	Bilingual Evaluation Understudy
METEOR	Metric for Evaluation of Translation with Explicit ORdering
BERTScore	Contextual Embedding-based Evaluation Metric
WER	Word Error Rate
RTF	Real-Time Factor
OAuth 2.0	Open Authorization Protocol 2.0
JWT	JSON Web Token
GDPR	General Data Protection Regulation
RFC	Request for Comments
ISO/IEC	International Organization for Standardization / International Electrotechnical Commission
IEEE	Institute of Electrical and Electronics Engineers

INTRODUCTION

1.1 Project Overview

Daily existence is marked by a plethora of interactions, ranging from casual exchanges to substantial articulations of viewpoints. However, the retention of every detail from these conversations poses a significant challenge, and crucial information is prone to being disregarded. As a result, this can engender miscommunication, the loss of opportunities, or the forfeiture of valuable insights. Manual note-taking often proves impractical, and an effective method for tracking key points across numerous interactions throughout the day is absent.

To address this issue, a smart conversation analysis device is suggested, intended to record, transcribe, and process spoken interactions. Advanced speech recognition and natural language processing techniques are utilized to identify key topics, sentiments, and relevant information from conversations.

A structured daily summary is subsequently produced, thereby furnishing an easily retrievable record of interactions. Furthermore, a productivity analysis component is incorporated to assess the efficacy of conversations by pinpointing action items, levels of engagement, and decision-making patterns. Consequently, evaluations can be conducted to determine the significance of discussions, facilitating the optimization of communication strategies.

This solution is engineered for efficiency, seamlessness, and user-friendliness, enabling integration into daily routines to augment productivity, memory retention, and communication precision. Regardless of whether it is employed in personal or professional contexts, a dependable methodology for tracking and analyzing conversations is offered, assisting users in making more informed decisions and enhancing time management. By streamlining information retrieval and augmenting overall efficiency, this project constitutes a notable advancement in the optimization of human interactions.

1.2 Need Analysis

In the contemporary, rapidly evolving environment, heightened competition compels both individuals and organizations to enhance productivity and refine time management practices. A significant impediment to effectively evaluating time utilization, however, is the absence of a structured methodology for monitoring and assessing daily activities. Daily routines encompass numerous interactions, meetings, and tasks. Without adequate tracking mechanisms, crucial information may be overlooked, resulting in inefficiencies, missed opportunities, and challenges in prioritizing essential tasks. This issue is mitigated by a sophisticated system engineered to observe daily interactions, assess productivity levels, and evaluate emotional states.

In light of the escalating emphasis on mental health and stress reduction, a parallel trend emerges, wherein a harmonious lifestyle is increasingly prioritized alongside career advancement. The identification and subsequent analysis of daily emotional variations provide crucial insights into prevailing emotional conditions. Consequently, by understanding the effects of various interactions on both mood and productivity, individuals are enabled to make informed choices, thus refining their daily routines, optimizing time management, and mitigating superfluous stressors. This heightened self-awareness cultivates proactive strategies aimed at establishing a more advantageous work-life equilibrium, which, in turn, contributes to improved efficiency and an enhanced overall quality of life.

This solution offers a complete package, combining real-time tracking, sentiment analysis, and productivity evaluation into a single, unified system. It allows for a more nuanced understanding of daily habits, which in turn facilitates better task prioritization and data-driven adjustments to work routines. Whether the goal is personal improvement, career advancement, or just a more manageable schedule, this innovation acts as a potent tool for streamlining daily activities and boosting performance. In our current, fast-moving world, where time is always of the essence, the demand for a smart, data-informed assistant to improve organization, decision-making, and overall productivity is more pressing than ever.

1.3 Research Gaps

Based on a review of the existing literature [1]-[12] and an examination of current systems, several significant research deficiencies have been discerned:

1. Absence of a Comprehensive Framework

Existing research on Automatic Speech Recognition (ASR) [1], [2], [8], text summarization [3], [4], [10], [12], and sentiment analysis [5]-[7] has approached these tasks in isolation. Consequently, no integrated system has been developed that consolidates transcription, summarization, and sentiment analysis within a single, productivity-oriented framework.

2. Deficiencies in Productivity Assessment

Current ASR tools and summarization models offer transcripts and summaries, yet they do not generate structured productivity metrics, such as engagement levels, decision-making efficiency, or actionable insights [9], [11].

3. Contextual Constraints in Summarization

Extractive models maintain contextual integrity but lack coherence [3], whereas abstractive models produce fluent summaries but frequently sacrifice critical meaning [4], [12].

Dialogue-focused summarization datasets, including QMSum [10] and SAMSum [12], demonstrate deficiencies in effectively managing conversational structures.

4. Difficulties in Implicit Sentiment and Emotion Detection

Sentiment models, such as BERT, RoBERTa, and DeBERTa [5]-[7], exhibit strong performance with explicit labels; however, they encounter challenges in recognizing implicit sentiment, sarcasm, and subtle emotional nuances, all of which are critical for assessing productivity.

5. Scalability and Real-Time Processing

Models that perform well, such as Whisper [2], [8], and PEGASUS [4], necessitate substantial computational resources. Furthermore, real-time, on-device deployment on lightweight hardware is still largely unexamined, thereby restricting accessibility in

environments with limited resources.

6. Cost and Accessibility Constraints

Cloud-based automatic speech recognition (ASR) and natural language processing (NLP) APIs, such as the Google API [2], present ongoing financial obligations, whereas open-source alternatives necessitate technical proficiency for implementation. Furthermore, there is a paucity of research focused on cost-effective, user-friendly systems suitable for adoption by both individuals and institutions.

7. Privacy and Security Considerations

The majority of current systems depend on cloud infrastructure for the processing of sensitive dialogues. Conversely, research has not sufficiently explored privacy-preserving architectures or secure local/hybrid deployment methodologies [9], [11].

1.4 Problem Definition and Scope

Problem Definition

Effectively managing spoken interactions within academic and professional settings presents a persistent challenge, stemming from several key factors:

- Information is often lost when discussions are not recorded or when notes are taken manually.
- Existing speech-to-text systems typically generate only unstructured transcripts, lacking structured insights.
- Most conversational AI systems, such as Alexa, Siri, and Google Assistant, prioritize task execution over outcome assessment, resulting in a lack of productivity evaluation.
- Current tools exhibit limited contextual awareness, failing to incorporate sentiment analysis, mood detection, and summarization capabilities.

Consequently, a centralized intelligent system is clearly needed, one that not only captures voice conversations but also processes them into structured outputs. These outputs should

provide real-time transcription, mood detection, productivity analysis, and summarization.

Scope of the Project

The Deeplogix project seeks to create a smart system that connects voice logging with productivity analysis. The project's scope encompasses:

- **Audio Acquisition and Transmission**

- An ESP32-S3 microcontroller interfaced with an INMP441 I2S digital microphone allows for real-time voice recording.
- WAV-formatted recorded audio is sent wirelessly via Wi-Fi to the backend server via HTTP-based communication.

- **Data Processing and Analysis**

- Display of transcribed conversations in a clear and readable format.
- Presentation of emotional tone analysis derived from sentiment detection.
- Visualization of productivity scores based on analyzed speech patterns.
- Delivery of summarized insights to help users quickly review discussions.

User Interface:

The mobile app will present users with transcripts, emotional tone analysis, productivity scores, and summarized insights, all in a user-friendly format.

System Characteristics:

The hardware will be designed to be lightweight and portable. Processing will occur on a local server to protect data privacy and lessen dependence on cloud services. The framework will be modular, allowing for future expansion to include features like multilingual support and cloud deployment.

Exclusions:

The current version does not include advanced multilingual capabilities, complete offline speech-to-text models, or enterprise-level cloud hosting, though these are planned for future updates.

1.5 Assumptions and Constraints

Assumptions

- It is assumed that users will consent to the recording and subsequent analysis of their conversations.
- Furthermore, the ESP32 microcontroller and INMP441 microphone are expected to consistently and accurately capture audio in real-time.
- End-users, including students, faculty, and professionals, are presumed to possess a fundamental level of digital literacy sufficient for mobile application operation.
- Finally, internet connectivity is assumed to be available for the processing of speech-to-text and natural language processing (NLP) models.

Constraints

- The system's functionality is constrained by the requirement for stable internet connectivity.
- The initial version of the application will be restricted to conversations conducted exclusively in the English language.
- Processing speed in real-time may be subject to variability, contingent upon the hardware limitations of both the ESP32 microcontroller and the server machine.
- Due to temporal limitations, advanced features such as offline speech recognition, multilingual support, and deep learning-based productivity scoring are scheduled for inclusion in subsequent versions.

1.6 Standards

The development of Deeplogix, which integrates speech recognition, text summarization, and sentiment analysis, demands adherence to established industry and research standards

to ensure interoperability, security, reliability, and ethical compliance. These relevant standards are categorized as follows.

1.6.1 Speech Recognition Standards

The IEEE Automatic Speech Recognition Guidelines furnish a structure for uniform evaluation metrics, encompassing Word Error Rate (WER), Real-Time Factor (RTF), and Signal-to-Noise Ratio .

Moreover, the ITU-T P.85 standard specifies methodologies for the subjective evaluation of speech quality within telecommunication-based automatic speech recognition systems.

Multilingual benchmarking datasets are widely employed to evaluate transcription accuracy across various languages and accents.

1.6.2 Natural Language Processing Standards

ROUGE and BLEU are established metrics used to evaluate the efficacy of text summarization and machine translation systems.

In the context of abstractive summarization, sophisticated evaluation metrics such as BERTScore and METEOR are utilized to gauge semantic similarity, surpassing the limitations of straightforward word matching.

ISO/IEC 2382-37:2017 (Information Technology — Vocabulary — Natural Language Processing) provides standardized definitions, thereby fostering uniform terminology within the field of NLP research.

1.6.3 Sentiment and Emotion Analysis Standards

SemEval benchmarks provide internationally recognized datasets and shared tasks, which aid in the evaluation of sentiment analysis and aspect-based sentiment analysis models.

Emotion annotation frameworks, including Ekman's Six Basic Emotions and Plutchik's Wheel, are frequently employed standards for labeling emotions in affective computing research.

1.6.4 Software Development and Security Standards

ISO/IEC 27001 provides a framework for information security management, thereby governing the secure processing and storage of sensitive conversational data.

Authentication and authorization mechanisms within both research prototypes and operational systems are implemented using established protocols, notably OAuth 2.0 and JSON Web Token (JWT, RFC 7519).

Moreover, the General Data Protection Regulation (GDPR) stipulates data privacy and protection mandates applicable to systems that handle user-generated conversational data.

1.6.5 Research Data and Reproducibility Standards

The FAIR principles (Findable, Accessible, Interoperable, Reusable) furnish a set of guidelines for research data management, with the objective of fostering transparency, facilitating reuse, and ensuring long-term accessibility.

IEEE DataPort and the ACM Digital Library establish standard repositories and practices for disseminating datasets, experimental findings, and reproducible research artifacts.

1.7 Approved Objectives

- A system that captures and processes speech
- An Android app that displays transcriptions, mood analysis, and productivity insights.
- A server that converts speech to text and analyzes conversations.
- A fully tested and verified system for accurate results.

1.8 Methodology

The DeepLogix project follows an Agile Software Development Methodology, emphasizing iterative development, modular design, incremental enhancements, and continuous feedback. This methodology supports gradual system evolution, allowing improvements and refinements based on testing results and user feedback while maintaining flexibility in both hardware and software development.

The initial phase focuses on requirement gathering and analysis. This involves studying existing voice logging systems, speech-to-text applications, and productivity or reflection tools to understand their capabilities and limitations. The analysis reveals several gaps in current solutions, such as limited integration between voice capture and structured analytics, lack of chronological organization, and minimal support for reflective insights such as summarization and engagement indicators. These observations guide the definition of functional and non-functional requirements for DeepLogix.

The next phase involves system planning and component selection, covering both hardware and software elements. Hardware components include the ESP32-S3 microcontroller, INMP441 I2S digital microphone, micro SD card for local audio storage, and a Li-ion battery for portable operation. On the software side, the system is planned around a Flask-based backend, a Flutter mobile application, Firebase services for storage and authentication, and local AI models executed using Ollama.

Following this, the system design phase is carried out, addressing hardware and software architecture in parallel. The hardware design integrates the ESP32-S3 with the I2S microphone to enable high-quality audio capture, local WAV file storage, and Wi-Fi-based audio transmission using HTTP. The software architecture is divided into backend and frontend layers. The backend, implemented using Flask, handles audio reception, speech-to-text processing, data storage, and API exposure. The frontend, developed using Flutter, provides visualization of transcriptions, summaries, and user insights. During this phase, system block diagrams, data flow diagrams, activity diagrams, and UI wireframes are designed to clearly define system structure and data movement.

The development phase involves implementing all planned components. On the hardware side, the ESP32 is programmed to perform real-time I2S audio capture, WAV file generation, SD card storage, and Wi-Fi-based file upload. The backend development includes REST API creation, audio file handling, speech-to-text conversion, and database integration. Local AI models accessed via Ollama are used for conversational interaction and summarization to maintain data privacy. The mobile application is developed using Flutter and Dart, following standard UI design practices and integrating real-time database synchronization.

The testing phase validates system functionality and reliability at multiple levels. Hardware testing evaluates microphone input quality, recording accuracy, storage reliability, and Wi-Fi transmission stability. Software testing includes unit and integration testing of backend APIs, verification of speech-to-text output, and validation of database synchronization. The mobile application undergoes functional and usability testing to ensure correct data display and smooth user interaction. Testing is performed at a prototype level to identify logical and integration issues rather than performance benchmarking.

The deployment phase involves running the backend on a local system and exposing it externally using a tunneling service for controlled access. The mobile application is deployed on Android devices for demonstration and testing. Pilot usage is conducted with a small group of users to observe real-world interaction patterns and system behavior.

The final phase focuses on maintenance and future enhancements. Planned improvements include enhanced speech feature extraction, multilingual support, improved NLP-based analytics, scalable cloud deployment, and partial on-device processing to reduce dependency on continuous network connectivity. These enhancements aim to extend DeepLogix from a functional prototype to a more scalable and robust intelligent voice analysis platform.

1.9 Project Outcomes and Deliverables

Expected Outcomes

- A working voice-logging device integrated with ESP32 and INMP441.
- Real-time transcription of recorded conversations.
- Mood detection and productivity analysis from speech.
- Summarized reports of daily conversations.
- A mobile app interface for users to view logs, mood, and productivity scores.

Deliverables

- Prototype hardware device.
- Backend server for processing audio.
- Mobile app (Flutter) for displaying outputs.
- Technical documentation (system design, workflow, APIs).
- Demo report/dashboard showing sample user outputs.

1.10 Novelty of Work

DeepLogix distinguishes itself through the tight integration of lightweight hardware and software components into a single, cohesive system for voice-based reflection and productivity analysis. Unlike many existing solutions that handle voice recording, transcription, and analysis as isolated functionalities, DeepLogix unifies these stages into a continuous end-to-end pipeline.

The system captures voice data directly at the hardware level using an ESP32-S3 microcontroller and an INMP441 digital microphone, ensuring controlled and efficient audio acquisition. Instead of functioning as a passive recording device, the hardware actively participates in the data pipeline by transmitting recorded audio over Wi-Fi to a backend processing system. This design eliminates dependency on manual uploads and simplifies user interaction.

A key novelty lies in how DeepLogix treats voice data. Rather than storing long raw audio recordings, the system converts speech into structured text, enabling chronological organization, keyword analysis, and summarization. This approach reduces storage

overhead while significantly improving usability and accessibility of recorded information.

In addition to transcription, the system applies heuristic-based analytical techniques to extract indicators related to communication patterns and engagement levels. Daily summaries and keyword trends provide users with reflective insights into their conversations, supporting awareness rather than absolute evaluation. These analytics are designed to be lightweight and interpretable, making them suitable for a prototype system.

Another important contribution is the emphasis on privacy-aware design. Conversational interaction and summarization are performed using locally executed language models, ensuring that sensitive user data is not shared with third-party cloud AI services. This design choice differentiates DeepLogix from many existing voice analytics platforms that rely heavily on external cloud-based processing.

Overall, DeepLogix moves beyond the concept of a conventional voice recorder. Through its compact hardware design, modular software architecture, and integrated analytical workflow, it functions as a personalized voice-based reflection and productivity support system. The project demonstrates how affordable IoT hardware combined with modern AI techniques can be used to build scalable and privacy-conscious intelligent applications.

Requirement Analysis

2.1 Literature Review

2.1.1 Theoretical Frameworks

The Deeplogix system's theoretical foundation is derived from a range of interdisciplinary domains, including speech processing, natural language understanding, affective computing, human-computer interaction, and decision support systems. These theoretical perspectives collectively shape the design and functional attributes of intelligent systems that analyze spoken dialogues to derive productivity and behavioral insights. Automatic Speech Recognition (ASR) Theory serves as the core principle of Deeplogix, primarily because it focuses on converting spoken language into a machine-readable format. ASR systems are designed to decode acoustic signals, phonetic components, and linguistic structures, thus enabling accurate speech transcription. The extant literature identifies several significant hurdles, including the mitigation of background noise, the accommodation of speaker accent variations, and the attainment of real-time transcription with minimal delay. These foundational theories underpin contemporary voice assistants and transcription systems, proving indispensable for subsequent applications such as sentiment analysis and summarization [1], [2]. Text Summarization Theory is of paramount importance in the process of distilling extensive conversational transcripts into coherent and succinct summaries. Extractive summarization methodologies operate by selecting salient sentences directly from the source text, whereas abstractive summarization techniques formulate novel phrases to encapsulate the fundamental meaning of the material.

Recent investigations underscore the efficacy of hybrid models, which integrate extractive and abstractive techniques, in enhancing the coherence, relevance, and readability of summaries, especially when applied to conversational data [3], [4], [10], [12].

Sentiment analysis and affective computing theory focus on identifying emotions, sentiment polarity, tone, and user intent in textual data. This area goes beyond simply classifying text as positive or negative by using detailed emotion detection methods, such

as Aspect-Based Sentiment Analysis (ABSA) These theoretical frameworks are particularly relevant to Deeplogix, as emotional and sentiment cues from conversations provide valuable insights into user mood, engagement, and overall productivity trends [5]-[7].

Human-Computer Interaction (HCI) and Productivity Theory suggest that intelligent systems should go beyond just collecting data passively. Instead, they should actively help users by providing structured feedback and actionable insights.

Research in this area highlights the importance of usability, accessibility, and seamless integration into daily routines to encourage wider adoption and ongoing interaction. These tenets guide the design of Deeplogix as a user-centric system, emphasizing the provision of relevant productivity and emotional feedback rather than the display of raw conversational logs [9], [11].

Knowledge Management and Decision Support System (DSS) Theory underscores the need for the systematic gathering, structuring, and retrieval of unstructured data, including conversational transcripts. DSS frameworks propose the use of analytical dashboards and summarized insights to aid users in making informed decisions. Therefore, within the context of Deeplogix, these theories support the transformation of unprocessed speech data into structured knowledge, thus facilitating personal reflection, productivity evaluation, and behavioral improvement.

2.1.2 Current Systems and Their Shortcomings

A few existing solutions offer partial answers to the problem, though they come with drawbacks:

1. Google Speech-to-Text / Whisper

- Advantages: High transcription accuracy and support for multiple languages.
- Drawbacks: Whisper's speed is an issue, and the Google API can be expensive at scale, plus it's susceptible to errors in noisy environments [1], [2], [8].

2. Voice Assistants

- Advantages: They handle real-time speech recognition and can execute tasks.
- Drawbacks: They don't assess productivity or generate detailed summaries.

3. Summarization Models

These models are adept at generating coherent summaries when applied to structured datasets. However, they encounter difficulties when processing conversational data and maintaining dialogue flow [3], [4], [10], [12].

4. Sentiment Analysis Models

These models excel in accurately classifying emotions and aspects within structured domains. Conversely, their performance diminishes when dealing with implicit sentiment and productivity contexts [5]-[7].

5. Hybrid Summarization & Productivity Tools

These tools leverage a combination of extractive and abstractive models to enhance coherence. Nevertheless, they are limited by their restricted deployment and lack of hardware integration.

Gap Identified:

- No existing solution integrates ASR, summarization, and sentiment analysis in a unified system.
- Current tools provide either raw transcripts, summaries, or emotions but not combined productivity insights.
- Most solutions are costly, cloud-dependent, or inaccessible for real-time personal/professional use.

2.1.1 Research Findings from Existing Literature

Table 2.1.1: Research Findings

S.No.	Team Member	Authors / Researchers	Paper Title	Tools / Technology Used	Findings	Citation No.
1	Aditya Tejpai	S. T. Ramacharla et al.	Speech to Text Transcription	Python, SpeechRecognition, PyDub	Developed ASR system with multi-source input and web-based access.	[1]
2	Aditya Tejpai	J. K. Höhne et al.	Automatic Speech-to-Text Transcription	Google API, Whisper	Whisper gave higher accuracy; Google API was faster but error-prone.	[2]
3	Aditya Tejpai	Z. A. Merrouni et al.	EXABSUM: Hybrid Summarization	TR-ISF, Graph Compression	Outperformed extractive models; coherence improved but context loss in strict limits.	[3]
4	Arpit Aggarwal	P. K. Biswas	Hybrid Chat Summarization	PEGASUS, BART, T5	Hybrid approach; PEGASUS performed best on 160k chat transcripts.	[4]

5	Arpit Aggarwal	N.Shafikuzzaman et al.	Sentiment Analysis in Software Eng.	Pretrained LMs (seBERT, RoBERTa)	Domain-specific PLMs best with large data; few-shot better with limited data.	[5]
6	Arpit Aggarwal	N. Mughal et al.	Aspect-Based Sentiment Analysis	DeBERTa, PaLM, GPT models	DeBERTa best for ATSA; LLMs strong but weak in implicit sentiment detection.	[6]
7	Harsh Lakyan	P. Durga & D. Godavarthi	Integrated Sentiment Analysis Model	BERT-large, D-RNN, SGD	Outperformed baselines on Twitter, Laptop, Restaurant datasets.	[7]
8	Harsh Lakyan	A. Radford et al.	Robust Speech Recognition with Whisper	Large-scale ASR pretraining	Trained on 680k hours; robust to noise and accents; strong zero-shot performance.	[8]
9	Harsh Lakyan	H. Bredin	pyannote.audio: Speaker Diarization	Segmentation + Embedding + Clustering	Open-source diarization pipeline; accurate speaker separation in real conversations.	[9]

10	Pankaj Sheokand	M. Zhong et al.	QMSum: Query-based Meeting Summarization	Query-focused summarization dataset	1,808 query-summary pairs; locate-then-summarize approach effective for meetings.	[10]
11	Pankaj Sheokand	J. Carletta et al.	AMI Meeting Corpus	Multimodal meeting dataset	100-hour annotated corpus for ASR, diarization, and meeting summarization tasks.	[11]
12	Pankaj Sheokand	B. Gliwa et al.	SAMSum: Dialogue Summarization Dataset	16k messenger-style dialogues	Benchmark dataset; highlights gap in dialogue-specific summarization models.	[12]

2.1.4 Problem Identification

The literature and existing systems reveal several key problems:

- Functionalities are fragmented; current systems usually handle transcription, summarization, or sentiment analysis separately [1]-[4].
- There's a lack of structured productivity evaluation; no solution provides organized productivity metrics based on conversations [5]-[7].
- Contextual challenges exist; summarization models struggle with unstructured dialogue and subtle meaning [3], [4], [10], [12].
- Implicit emotion detection is often inadequate; sentiment models frequently miss

subtle emotional cues [6], [7].

- Accessibility issues are common; many tools are expensive, depend on cloud services, and aren't designed for low-cost portable hardware [2], [8].

2.1.5 Survey of Tools and Technologies Used

Prior investigations and practical implementations have explored a variety of tools and technologies designed to support speech recognition, text summarization, sentiment analysis, and system integration. These tools form the foundational technical elements for systems similar to Deeplogix, each possessing unique strengths and weaknesses.

Automatic Speech Recognition (ASR) tools, such as Google Cloud Speech-to-Text, Whisper, and Vosk, are commonly utilized to convert spoken language into text. These tools are acknowledged for their reliable transcription capabilities across a wide range of languages and accents. However, they differ significantly in terms of processing speed, transcription accuracy in noisy settings, and their appropriateness for real-time or on-device deployment.

While cloud-based solutions offer high accuracy, they often introduce latency and cost constraints, whereas offline models face challenges related to computational efficiency [1], [2], [8].

Text Summarization Tools include extractive approaches like TextRank and abstractive models such as PEGASUS, T5, and BART, along with hybrid techniques such as EXABSUM. Abstractive summarization models are generally more effective at capturing contextual meaning and generating human-like summaries, particularly for long-form text. Hybrid approaches further improve coherence by combining extractive sentence selection with abstractive generation. Despite their advantages, these models often struggle with conversational data and require substantial computational resources, which limits their use in lightweight or real-time systems [3], [4], [10], [12].

Sentiment Analysis Tools are primarily based on transformer architectures, including

BERT, RoBERTa, DeBERTa, seBERT, PaLM, and GPT-based large language models. These instruments have demonstrated significant effectiveness in the classification of emotion and sentiment, especially when tailored to particular areas of application. Studies indicate that models refined for specific domains typically outperform general-purpose sentiment models, particularly in the analysis of nuanced emotional cues and contextual sentiment related to productivity and engagement [5]-[7].

Frameworks and libraries are indispensable for system construction and integration. Tools such as HuggingFace Transformers, SpeechRecognition, PyDub, Flask or FastAPI for backend development, and Flutter for mobile application development provide modular, open-source, and scalable solutions. These frameworks support model integration, audio processing, backend communication, and cross-platform user interface development, thus making them suitable for both rapid prototyping and research-focused systems [9], [11].

Despite the availability of these resources, current methodologies have several limitations. Abstractive summarization models and large pretrained language models often require significant computational power, making them unsuitable for real-time use or situations with limited resources [3], [4], [12]. Although Whisper is quite accurate, it isn't fast enough for continuous real-time transcription [2], [8]. Moreover, freely available APIs usually have usage limits, while commercial APIs involve substantial operational costs. Most importantly, existing tools and techniques aren't designed for integrated, on-device deployment using specialized hardware, highlighting the need for a unified and efficient solution, like Deeplogix.

2.2 Software Requirement Specification

2.2.1.1 Purpose

Deeplogix is conceived as an intelligent, automated system. Its primary function involves recording conversations through voice input, subsequently converting these into text format, and then analyzing the emotional tone, productivity level, and principal topics discussed. The system's objective is to assist users in monitoring their daily conversations, identifying productivity patterns, and generating succinct reports for self-evaluation and performance improvement. Through the integration of speech-to-text technology, sentiment analysis, and summarization functionalities, Deeplogix converts raw audio data into practical insights.

2.2.1.2 Intended Audience and Reading Suggestions

- Professionals & Working Individuals - To track meetings, conversations, and productivity while receiving concise summaries and insights.
- Students & Researchers - To analyze study sessions, discussions, and generate summarized notes for better understanding.
- Educators & Trainers - To review lecture or training session content, monitor learner engagement, and provide feedback more effectively.

2.2.1.3 Project Scope

The system will provide a secure login for users. It will record and process real-time audio from conversations, using the ESP32-based hardware and a mobile application. In addition, it will automatically convert speech to text and securely store the resulting transcriptions. The system will also analyze conversations to assess mood, emotional state, and productivity. Finally, it will generate concise summaries and key insights from daily recordings.

2.2.2 Overall Description

2.2.2.1 Product Perspective

Deeplogix is a wearable-assisted system that bridges users and their productivity insights. It captures audio via ESP32 hardware, processes it through an AI server, and delivers mood, summary, and productivity feedback to the mobile app.

2.2.2.2 Product Features

- User registration & secure login
- Real-time voice capture via ESP32 & mobile app
- Automatic speech-to-text transcription
- Mood & productivity analysis
- Conversation summarization

2.2.3 External Interface Requirements

2.2.3.1 User Interfaces

- Mobile App: Provides access to summaries, insights, and analyses.
- Minimal Voice Interface: One-tap recording control (hardware button).

2.2.3.2 Hardware Interfaces

- ESP32-S3 with INMP441 I2S Microphone: Captures voice input using digital I2S communication.
- Micro SD Card Interface (SPI): Stores recorded audio locally in WAV format.
- Wi-Fi Interface (ESP32 Built-in): Uploads audio files directly to the backend server using HTTP.
- Backend Processing System (Laptop/Server): Performs speech-to-text conversion and AI-based analysis.

Minimum Mobile Device Requirements:

- 2 GB RAM
- Android 9 (Pie) or later

Server Requirements :

- 8 GB RAM
- Quad-core CPU
- 50 GB storage

2.2.4 Additional Non-functional Requirement**2.2.4.1 Performance Specifications**

The system's architecture is designed to accommodate a substantial user base concurrently, encompassing both demonstration and testing phases. Real-time speech transcription is anticipated to function with a latency ranging from 5 to 7 seconds per sentence, contingent upon hardware specifications and network conditions. Furthermore, the text summarization and mood analysis processes should be finalized within 15 to 20 seconds following the recording session's termination, thereby facilitating prompt user feedback with negligible delay.

2.2.4.2 Safety Considerations

To maintain consistent functionality despite fluctuating network conditions, the application will locally cache audio recordings on the device when internet access is unavailable, subsequently synchronizing this data upon the restoration of connectivity. To mitigate the risk of data loss, fundamental data backup protocols will be established utilizing complimentary cloud storage services. Moreover, resilient error-handling mechanisms are integrated to guarantee application stability and avert crashes during audio recording and processing tasks.

2.2.4.3 Security Specifications

The system employs fundamental security protocols to safeguard user data and uphold system integrity. User authentication is facilitated through a straightforward email and password-based login system. Stored audio files and their transcriptions are safeguarded through fundamental encryption methods, thereby mitigating the risk of unauthorized access. Furthermore, all interactions between the mobile application and the cloud-based

backend server are secured via HTTPS, ensuring the secure transmission of data.

2.3 Cost Evaluation

The cost evaluation of the Deeplogix prototype encompasses expenditures associated with hardware components, software services, and representative development activities. The system's architecture is deliberately structured to be both economical and accessible, leveraging readily obtainable hardware components and open-source software frameworks. A comprehensive cost breakdown is presented in Table 2.3.1, demonstrating the economic viability of the proposed solution.

1. Bill of Materials (BoM)

Table 2.3.1 Bills of material.

Component	Quantity	Cost per Unit (INR)	Total Cost (INR)
ESP32-S3 Mini	1	560	560
INMP441 I2S Microphone	1	240	240
microSD Card (16 GB)	1	170	170
microSD Card Reader Module	1	25	25
LiPo Battery (3.7V, 1000mAh)	1	200	200
3D Printed Enclosure	1	50	50
Miscellaneous (Cables, Soldering Supplies, etc.)	-	-	200
Non-Recurring R&D Materials	-	-	3980
Total Development Cost			5425

2. Software and Cloud Services

Table 2.3.2 Software and Cloud Services.

Software/Service	Usage/Limit	Estimated Cost (INR)
Google Speech-to-Text API	Free Tier (60 minutes/month)	0
HuggingFace Transformers (BERT, T5)	Local Use	0
Ngrok (For Remote Access Demo)	Free Plan	0
Total Software/Cloud Cost		0

Note: The project currently operates under the free tiers of all listed services. If scaled, cloud deployment and API costs may be incurred.

3. Development Cost

As this is a student project, no actual development costs are incurred. However, the following table 2.3.3 is an indicative estimate of effort for real-world consideration:

Table 2.3.3: Development Cost.

Task	Estimated Hours	Hourly Rate (INR)	Estimated Cost (INR)
Firmware Development (ESP32)	30	0	0
Backend Development (FastAPI)	40	0	0

Mobile App Development (Flutter)	40	0	0
Total Development Cost			0

4. Summary

Table 2.3.4: Total cost.

Category	Total Cost (INR)
Hardware	5425
Software/Cloud	0
Development	0
Total Project Cost	5425

5. Remarks

The Deeplogix prototype was built with inexpensive, easily sourced hardware, making it both affordable and simple to reproduce. Every software tool, framework, and API integrated into the system is either free or open-source, keeping ongoing operational costs low. The system's design is also built to scale in the future without dramatically raising costs, facilitating upgrades and the addition of new features as needed. The cost estimates provided don't account for possible future expenses related to cloud hosting or paid APIs, which could be integrated later, contingent on performance and scalability needs.

2.4 Risk Analysis

1. Technical Risks

The system's transcription accuracy could be compromised by background noise, simultaneous speech, or substandard audio quality during the recording phase. Furthermore, devices with limited resources or lower specifications might introduce processing delays in tasks like text summarization and mood analysis, potentially affecting the system's real-time functionality.

2. Operational Risks

The application's utility is significantly contingent upon the precision and pertinence of the generated summaries. Consequently, if the summaries or mood insights are inaccurate, users might find the application less valuable or dependable. Moreover, operational challenges could arise from insufficient storage capacity on user devices if recorded data is not managed effectively or deleted promptly.

3. Security Risks

The potential for data leakage exists if audio recordings or transcribed text stored locally are not sufficiently secured. Furthermore, the use of free third-party APIs could potentially compromise privacy and data integrity if their data handling protocols are not meticulously monitored and configured.

4. Mitigation Strategies

To mitigate these risks, noise reduction and fundamental audio preprocessing methods are utilized to enhance transcription accuracy. The application is designed to process shorter audio segments rather than extended continuous recordings, thereby decreasing computational demands and enhancing response times. Only essential data is retained, and all stored information is safeguarded through the use of standard encryption libraries. Moreover, clear and concise user instructions are provided to encourage the safe and responsible utilization of the system.

Methodology Adopted

Deeplogix's development is guided by a structured methodology, designed to facilitate the precise acquisition, processing, and analysis of voice conversations, thereby ensuring the fulfillment of project goals. This segment elucidates the investigative methodologies, the proposed solution, the work breakdown structure, and the tools and technologies utilized throughout the project.

3.1 Investigative Techniques

Before deciding on the approach shown in Table 3.1.1, we first evaluated several investigative techniques to create a system that combines hardware-based voice logging with AI-driven analysis. Three common investigative methods, often used in academic and engineering projects, are:

Table 3.1.1: Investigative Technique

S.No.	Investigative Technique	Description	Application in Deeplogix
1	Descriptive	Focuses on describing the existing problem, its causes, and current solutions.	Used to analyze existing speech-to-text systems, mood detection methods, and productivity tracking tools.
2	Comparative	Involves comparing existing methods, tools, or algorithms to identify the most effective ones.	Applied to compare speech recognition APIs (e.g., Google Speech-to-Text vs. Vosk) and summarization models (e.g., TextRank vs. T5).
3	Experimental	Based on designing experiments to test prototypes.	Implemented when integrating hardware (ESP32) with backend processing and mobile application for end-to-end testing.

3.2 Proposed Solution

Introduction

Conventional voice recording systems are limited to capturing and storing unprocessed audio, thereby failing to provide substantive insights. Deeplogix seeks to overcome this constraint by employing AI-driven transcription, sentiment analysis, and summarization techniques to furnish structured understandings of conversations and study sessions.

Objectives of Deeplogix

- Facilitate the recording of conversations and discussions via a mobile application.
- Convert spoken language into text to generate structured transcripts.
- Implement natural language processing (NLP) methodologies for mood detection and fundamental productivity analysis.
- Produce summarized text for expedited review.
- Present findings within a user-friendly mobile app dashboard.

System Features

- **Mobile Application:** Enable audio recording, transcript viewing, summary generation, and mood analysis.
- **Backend Processing :** This handles speech-to-text conversion, sentiment analysis, and summarization.
- **Analytics Dashboard:** Presents emotional trends and productivity patterns using straightforward charts.
- **Data Privacy:** User security is ensured through the use of free-tier cloud services with basic encryption.

Benefits of the Solution

- For Individuals: It aids in tracking discussions, meetings, and personal productivity.
- For Students & Researchers: Automatically creates summaries of study sessions or group discussions.

- For Educators/Institutions: It lessens the burden of manual note-taking and enhances accessibility to key discussion points.

Conclusion

Deeplogix fills the space between basic voice recorders and intelligent assistants, providing structured insights, boosting productivity, and making it easier to review recorded sessions.

3.3 Work Breakdown Structure

The initial phase of the project, Project Planning, encompasses the delineation of the project's scope, functional prerequisites, and criteria for success. This stage further entails the determination of the technological framework, specifically the adoption of Flutter for frontend development and either FastAPI or Firebase for backend services.

Subsequently, the System Design phase concentrates on the specification of the backend architecture, which involves the design of APIs dedicated to speech transcription, sentiment analysis, and text summarization. Following this, a machine learning pipeline is implemented to facilitate sequential data processing, progressing from speech-to-text conversion to sentiment analysis, summarization, and productivity scoring.

Furthermore, mobile UI/UX designs are developed to facilitate audio recording, playback, and the presentation of analytical results.

The Development Phase encompasses the construction of machine learning models. These models, engineered for speech-to-text conversion, sentiment analysis, and summarization, undergo training or refinement to enhance their efficacy on mid-range devices. Subsequently, these models are integrated into a unified processing pipeline and made available via backend APIs. Simultaneously, the mobile application is developed to manage audio recording, playback, and the visualization of insights, encompassing transcripts, emotional states, summaries, and productivity metrics.

Transcription accuracy is evaluated throughout the testing and validation phases,

considering variations in accents and the presence of background noise. Concurrently, the outputs of sentiment detection and summarization are validated using pre-existing benchmark datasets. Integration testing ensures the smooth operation of the entire pipeline, and the mobile application is subjected to performance and usability testing on mid-range Android devices.

Following this, deployment and maintenance involve the deployment of backend APIs on free-tier cloud platforms and the packaging of the Android application as an APK for demonstration purposes. After deployment, system performance is monitored, user feedback is gathered, and machine learning models are iteratively refined to improve the system's overall effectiveness.

3.4 Tools and Technology

Deeplogix was built using a mix of open-source and free-tier technologies. This approach keeps development expenses down while still allowing for future growth. The system's backend is built with Python, utilizing frameworks like Flask or FastAPI. These frameworks manage server-side logic, API requests, and the processing of speech transcription, sentiment analysis, and text summarization. Natural Language Processing tasks rely on open-source libraries such as HuggingFace Transformers, NLTK, and SpaCy. These libraries offer pre-trained models and tools that can be tailored to perform text analysis, sentiment detection, and summary generation with efficiency.

The mobile application is created using Flutter and Dart, allowing for a single codebase that can be deployed across both Android and iOS devices.

The application provides users with the capability to record audio, access generated transcripts, and engage with analyzed outcomes, including summaries and mood assessments.

Data storage is facilitated by lightweight database solutions, exemplified by Firebase, which accommodates both local and cloud-based storage of user data and processed

information Secure communication between the mobile application and the backend server is established via HTTPS, and basic authentication mechanisms are employed to safeguard user access During the development and testing phases, Ngrok is utilized to expose the local backend server for external access For deployment purposes, free-tier hosting platforms, such as Heroku, are employed Version control and collaborative development are managed using GitHub throughout the project's duration.

Design Specifications

4.1 System Architecture

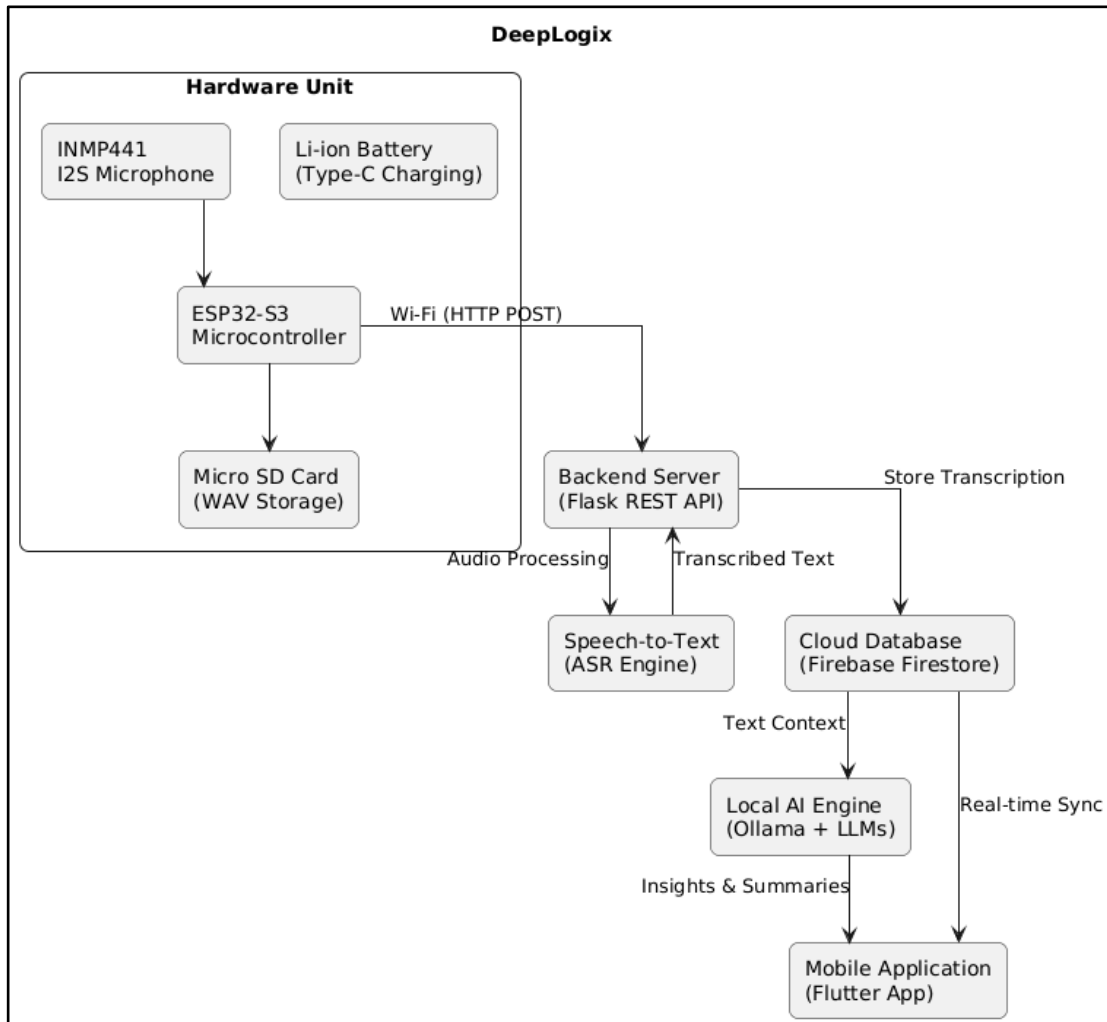


Figure 4.1.1 Block Diagram

Deeplogix's proposed system architecture is designed to seamlessly integrate hardware, communication, and software components, allowing for efficient voice logging and analysis. The hardware includes an ESP32 microcontroller connected to an INMP441 digital microphone, which captures real-time audio. This recorded audio is stored locally

in WAV format and transmitted directly to the backend server over a Wi-Fi connection using HTTP-based communication. This approach removes the dependency on intermediary relay devices while ensuring reliable and scalable data transmission.

The backend server, implemented using Flask and hosted on a laptop system, is exposed externally using Ngrok tunneling to facilitate controlled remote access during development and testing. Upon reception, the audio data undergoes multiple processing stages on the server. These include speech-to-text conversion, followed by text-based analytical processing such as heuristic productivity evaluation, keyword analysis, and conversation summarization. The processing pipeline is designed to operate modularly, enabling efficient handling of voice data while maintaining flexibility for future enhancements.

The processed outputs—including transcriptions, productivity indicators, and summarized content—are then stored in a cloud-based database and synchronized with the mobile application. The mobile application retrieves this information in real time and presents it through an intuitive user interface for visualization and review. This architecture, as depicted in Table 4.1.1, ensures seamless interaction among hardware, software, and communication components, thereby establishing Deeplogix as a dependable system for intelligent voice logging and productivity tracking.

4.2 Design Level Diagrams

1. Use Case Diagram

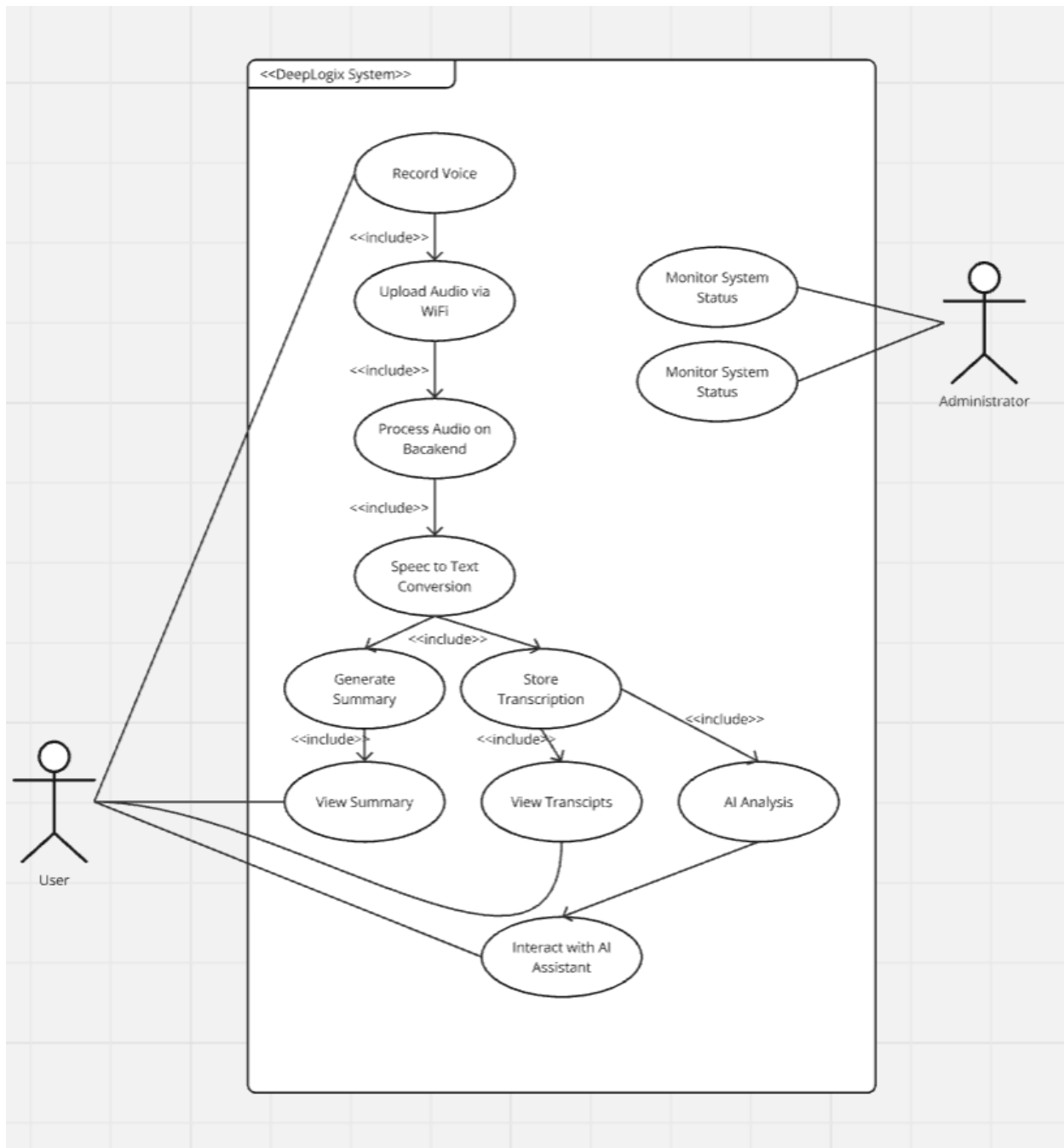


Figure 4.2.1 Use Case Diagram

Figure 4.2.1 illustrates the overall interaction between users, administrators, and the DeepLogix system. The workflow begins when the user records voice input using the hardware device. The recorded audio is transmitted to the system over a Wi-Fi connection and uploaded to the backend server for processing. The primary function of the system is to convert the received audio into text using speech-to-text processing.

Once the speech is converted into text, the system performs additional operations such as generating summaries, storing transcriptions, and enabling AI-based analysis. These processed results are then made available to the user through the mobile application, allowing them to view transcriptions, review summaries, and interact with the AI assistant for further insights.

From an administrative perspective, the administrator interacts with the system to monitor its status and manage users. These functions ensure smooth system operation and proper management of user activity. Overall, the use case diagram represents the complete interaction flow, showing how user input is processed by the system and transformed into meaningful insights for both users and administrators.

2. Activity Diagram



Figure 4.2.2 Activity Diagram

The activity diagram of the Deeplogix System, as shown in Figure 4.2.2, visually represents the process's sequential flow. The process starts with the ESP32 device capturing audio, which is then sent to the mobile application. The mobile application then uploads this data to the server for storage. The server then tries to convert the recorded speech into text. If this conversion is successful, a transcription is created; otherwise, the system either retries the process or notifies the user.

After transcription, the text undergoes several analytical procedures, including mood detection, productivity assessment, and summarization. These findings are then

integrated to produce the final results. Once generated, these results are sent back to the mobile application. The system handles any processing delays before delivering the output. As a result, the user receives the processed results directly within their mobile application.

3. Swimlane Diagram

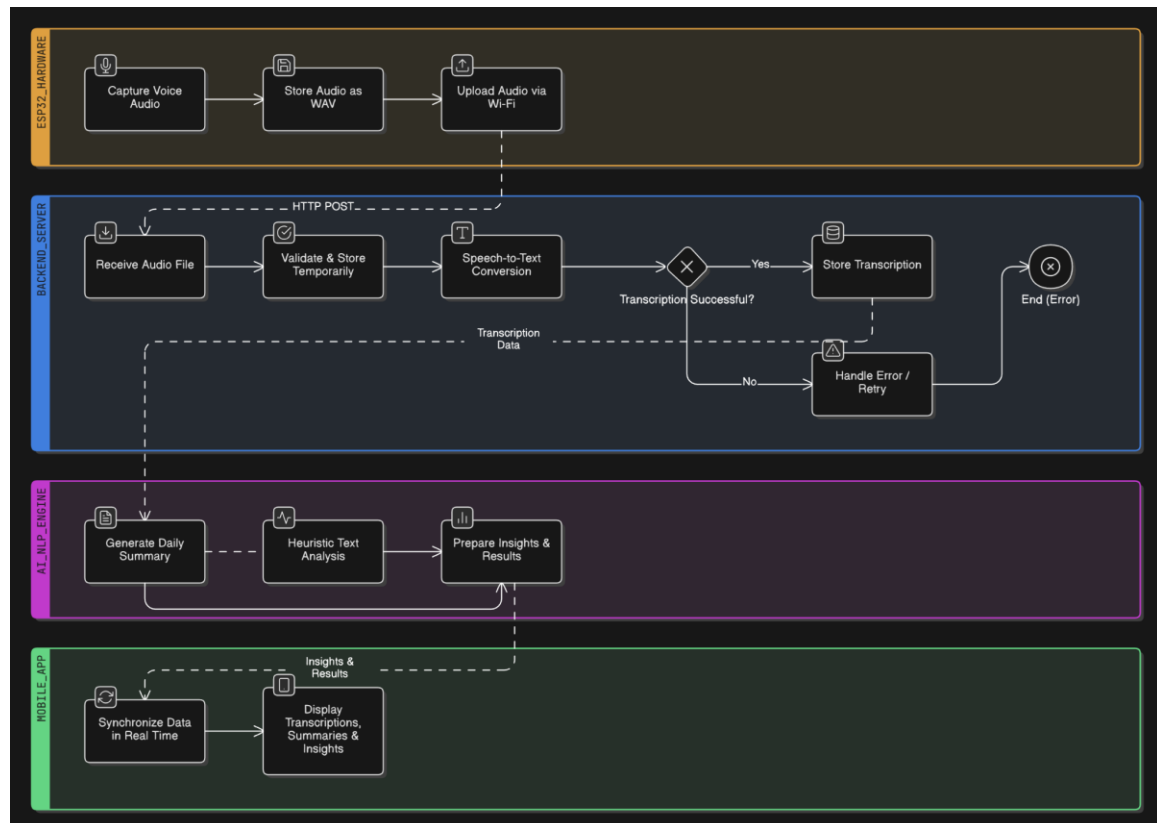


Figure 4.2.3 Swimlane Diagram

The swimlane diagram of the Deeplogix System, as illustrated in Figure 4.2.3, depicts the sequential interactions among its constituent elements: the ESP32, the Mobile App, the Server, and the NLP Engine. The process initiates with the ESP32 capturing audio data, which is subsequently transmitted to the mobile application. Upon receipt, the app uploads this audio to the server, where it undergoes both storage and conversion into textual format. Following a successful conversion..

If transcription isn't generated, the user is notified. The transcription then goes to the NLP engine. This engine analyzes the mood, checks productivity, and summarizes the text to create the final results. These results are sent back through the server to the

mobile app. The app handles any delays and then shows the results to the user. In short, Figure 3 shows how the different parts of the system work together, making the process clear from recording to the final output.

4.Data Flow Diagram

DFD level 0

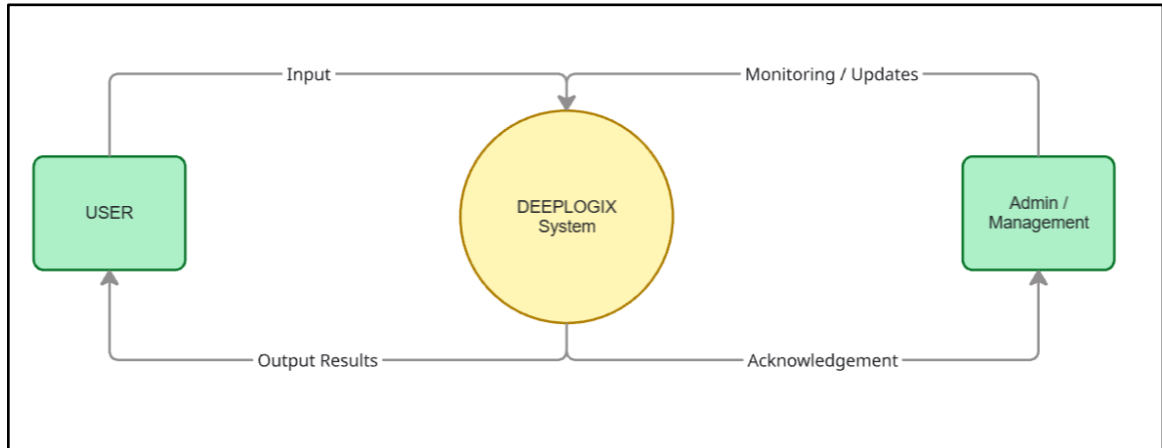


Figure 4.2.4 DFD level 0 Diagram

The Level 0 Data Flow Diagram (DFD) of the Deeplogix System (see Fig. 4) provides a high-level overview of the system's interactions with external entities. Users input data, including audio, and subsequently receive processed output. Conversely, the Admin/Management interface with the system by overseeing operations, updating information, and maintaining operational continuity. Furthermore, the system communicates acknowledgments to the admin in response to their actions.

DFD level 1

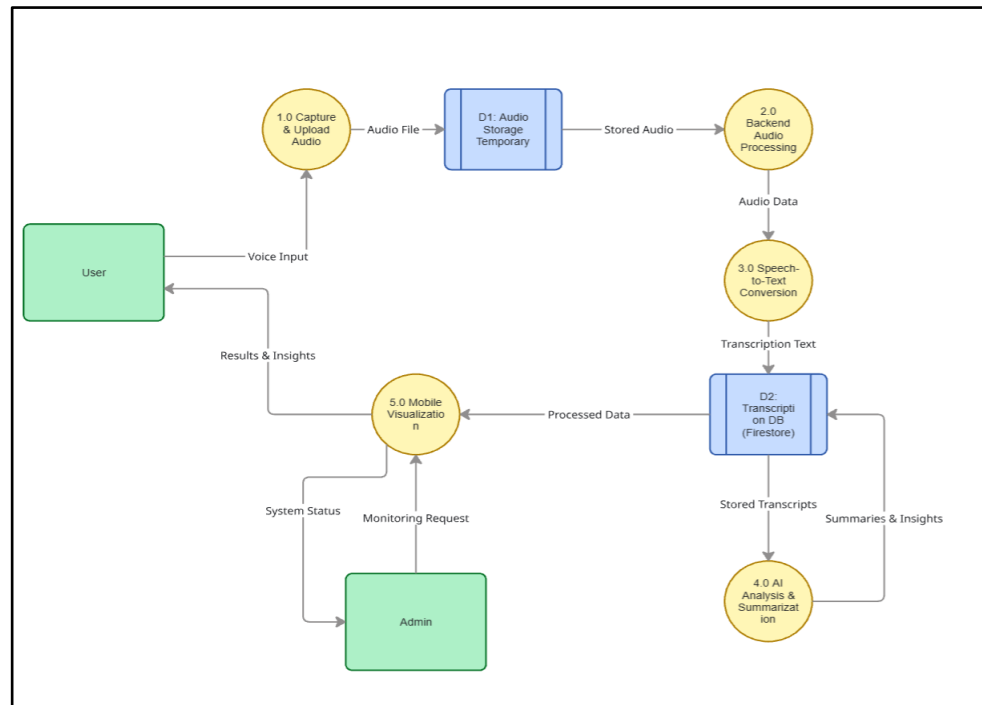


Figure 4.2.5 DFD level 1 Diagram

DFD Level 1 of the Deeplogix System (Fig. 5) provides a more granular view of the data's flow through the system's various elements. Initially, the user begins the process by recording their voice using the hardware microphone. This recorded file is then sent to the mobile application, which subsequently transmits it to the primary storage. The server, which handles all analytical functions, retrieves the file from storage. After the analysis is complete, the server sends the results back to the mobile app. The app then displays the processed results to the user. Concurrently, the administrator maintains constant supervision of the process, receiving updates and confirmations from both the mobile app and the server. This ensures the system's uninterrupted functionality and keeps the administrator informed of its current status.

DFD Level 2

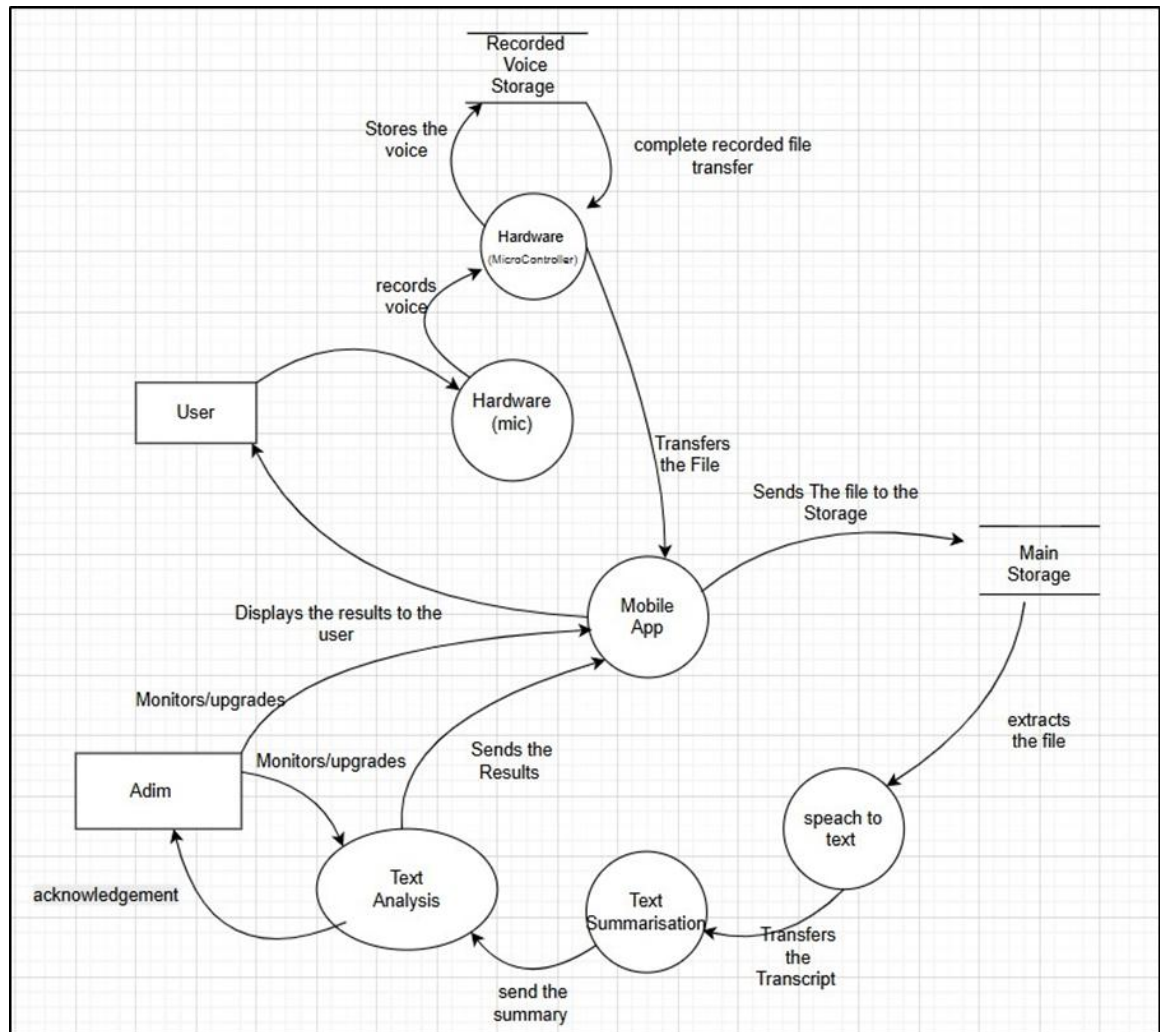


Figure 4.2.6 DFD level 2 Diagram

DFD Level 2 of the Deeplogix System (Fig. 6) provides a detailed view of the data flow. The user starts by recording their voice using the microphone. The microcontroller then saves and sends the recording to the mobile application. The file is then sent to the main storage, where it is extracted and converted from audio to text. After that, the transcript undergoes text summarization and analysis, leading to the creation of relevant results. These results are then sent back to the mobile app for the user to see. At the same time, the administrator monitors the entire process, makes updates as needed, and receives system notifications.

4.3 User Interface Diagram

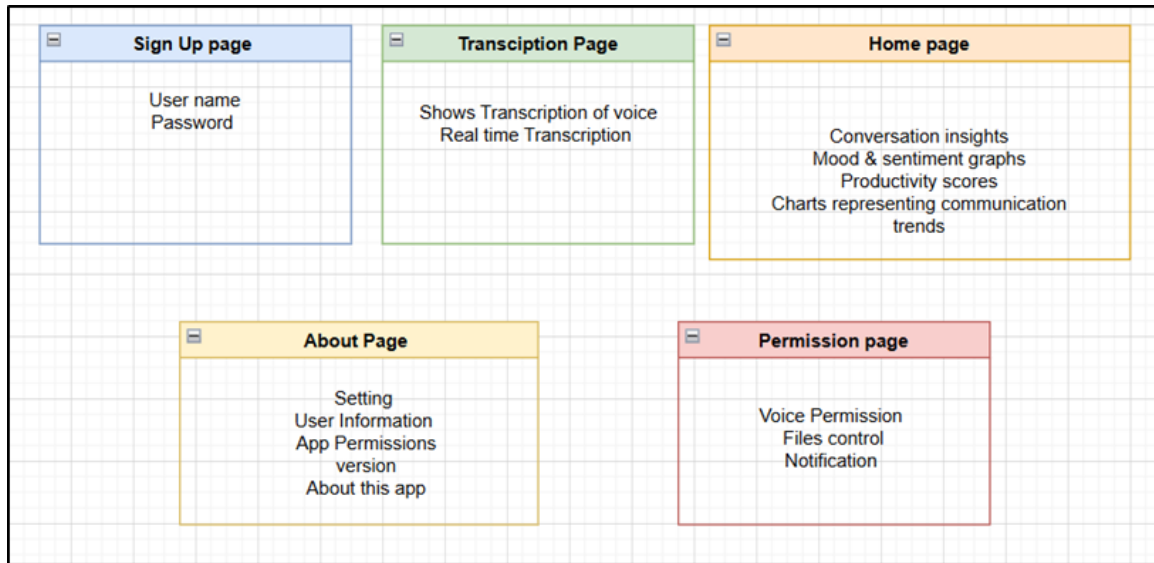


FIGURE 4.3.1. User Interface Diagram

The primary user interface pages of Deeplogix are illustrated in diagram 4.3.1. The Sign Up page facilitates user registration via a username and password. The Permission page governs access to the microphone, files, and notifications. The Transcription page enables real-time voice-to-text conversion. Furthermore, the Home page presents conversation insights, sentiment graphs, productivity scores, and trend charts. The About page furnishes information regarding settings, user data, app permissions, versioning, and the application's background. These screens collectively constitute the fundamental interaction flow of Deeplogix.

IMPLEMENTATION AND EXPERIMENTAL RESULTS

5.1 Experimental Configuration

The experimental configuration for Deeplogix was established to assess the entire operational sequence of the voice logging and analysis system. This configuration incorporated hardware for audio acquisition, a backend server for processing, and a mobile application for result presentation.

Audio was recorded using an ESP32 microcontroller interfaced with an INMP441 I2S microphone. The acquired speech data was subsequently transmitted to a mobile application via wifi. Brief audio samples, ranging from 5 to 15 seconds in duration, were recorded within typical indoor environments.

The backend server, which handled speech-to-text conversion, mood detection, productivity analysis, and text summarization, was implemented on a laptop system utilizing Flask/FastAPI. Ngrok was employed to expose the server for testing purposes. Speech recognition capabilities were provided by the Google Speech-to-Text API, while open-source models were utilized for the natural language processing tasks.

A Flutter-based Android application facilitated the reception of audio data, its subsequent upload to the server, and the display of the resulting transcription, mood assessment, productivity score, and summary. The testing phase concentrated on evaluating transcription accuracy, processing delay, and overall system stability.

This configuration successfully validated the functional integration of the hardware components, backend services, and the mobile application.

5.2 Experimental Analysis

5.2.1 Data

The experimental evaluation utilized voice recordings obtained during the testing phases. These recordings were produced by participants articulating English within typical indoor environments. Each audio sample exhibited a duration spanning from 5 to 15 seconds.

To facilitate compatibility with the speech recognition module, the recorded audio data was

preserved in WAV format. Subsequent to the speech-to-text conversion, the resultant textual data served as the input for mood detection, productivity analysis, and summarization tasks.

No external datasets were incorporated throughout the experimentation. All data was generated in real-time to assess the system's performance within realistic usage scenarios.

5.2.2 Performance Parameters

The performance of the DeepLogix system was assessed using multiple parameters to evaluate its reliability, efficiency, and quality of output across different stages of operation.

Transcription

Accuracy:

This parameter measures the correctness of the speech-to-text conversion by comparing the generated transcription with the original spoken content to ensure meaningful and understandable text output.

Processing

Latency:

This metric evaluates the time required to complete backend processing tasks, including speech-to-text conversion, text analysis, and summarization, after the audio data is received by the server.

System

Stability:

This parameter examines the reliability of Wi-Fi-based audio transmission, backend server execution, and the responsiveness of the mobile application during continuous operation.

Summary

Relevance:

This assessment focuses on the clarity, coherence, and usefulness of the automatically generated summaries, particularly for short and informal conversational inputs.

Productivity

Score

Consistency:

This metric analyzes the consistency of heuristic-based productivity indicators when similar speech samples are processed, ensuring that the scoring reflects relative trends rather than random variation.

5.3 Working of the Project

The DeepLogix system is a voice-based personal intelligence platform designed to capture user speech, convert it into structured digital information and provide meaningful insights through a mobile application. The working of the project involves the integration of hardware-based voice acquisition, backend software processing, cloud-based storage, and AI-driven analytics. This section explains the internal operation of the system in a structured and detailed manner.

5.3.1 Procedural Workflow

The procedural workflow of DeepLogix describes the complete end-to-end sequence of operations starting from voice capture to intelligent visualization. The system follows a modular and layered workflow to ensure reliability, scalability, and ease of maintenance.

Step-by-Step Workflow Description

- **System** **Initialization**
The hardware device initializes and connects to a predefined Wi-Fi network. Simultaneously, the backend server is kept active and exposed to the internet using a tunneling service.
- **Voice** **Capture**
The microphone, connected to the hardware, is employed to capture the user's spoken input. To maintain uniformity, the audio is digitally sampled and recorded for a specified duration.
- **Local** **Audio** **Storage**
The captured voice data is then stored locally on a Micro SD card, employing the WAV format. A compatible WAV header is included to ensure interoperability with speech recognition software.
- **Audio** **Transmission**
Subsequent to the recording phase, the audio file is transmitted to the backend server through an HTTP POST request. Multipart file transfer is utilized to ensure reliable transmission.

- **Backend**

The backend server validates the received audio file, examining its format, size, and integrity before any further processing occurs.

Validation
- **Speech-to-Text**

The validated audio is then forwarded to the speech recognition module. Ambient noise calibration is conducted, and the speech is then transcribed into textual form.

Conversion
- **Data**

The recognized text, together with relevant metadata including user identification and timestamps, is stored in a cloud-based database. This enables the chronological tracking of conversations.

Storage
- **Real-Time**

The mobile application persistently observes the database for modifications. Consequently, novel transcriptions are instantaneously integrated into the application, eliminating the necessity for manual updates.

Synchronization
- **AI-Driven**

Local AI models leverage the archived textual data to produce daily summaries, chatbot interactions, and heuristic-driven personal development assessments.

Analysis
- **User-Centric**

The mobile application displays transcriptions, summaries, and indicators in a manner that is both intuitive and easily navigable for the user.

Visualization

This procedural sequence guarantees the autonomous functionality of each individual component, while concurrently fostering a unified system performance as shown in figure 5.3.1.



FIGURE 5.3.1 :Workflow Diagram

5.3.2 Algorithmic Approaches Employed

The DeepLogix system employs a variety of algorithmic methodologies to analyze voice data and produce actionable insights. These algorithms are predominantly heuristic and prototype-based, thereby serving as a suitable means for academic demonstration.

Algorithm 1: Speech-to-Text Conversion Algorithm

Objective:

To transcribe recorded voice input into textual format.

Description:

The algorithm accepts a WAV audio file as its input, subsequently applies ambient noise reduction techniques, and executes speech recognition to produce text.

Pseudocode:

Input: WAV audio file

Output: Recognized text

Begin

 Load audio file

 Adjust for ambient noise

 If speech detected then

 Convert speech to text

 Return text

 Else

 Return empty string

End

Algorithm 2: Daily Summary Generation Algorithm

Objective:

To create a brief summary of daily conversations.

Description:

All transcriptions from a chosen date are combined and then processed by a local language model, which produces a short summary.

Pseudocode:

Input: Daily transcriptions

Output: Summary text

Begin

 Collect all transcriptions for the day

 Preprocess text

 Pass text to language model

 Generate summary

 Return summary

End

Algorithm 3: Personal Development (PD) Score Algorithm**Objective:**

To calculate a comparative score that reflects the depth of communication.

Description:

This algorithm assesses both the variety of vocabulary used and the significance of keywords pertinent to the individual's profession. The resulting score is then normalized to facilitate comparative analysis.

Pseudocode:

Input: Transcribed text, profession keywords

Output: PD score

Begin

 Count unique words

```

Match profession keywords
Compute weighted score
Normalize score
Return PD score
End

```

5.3.3 Project Deployment

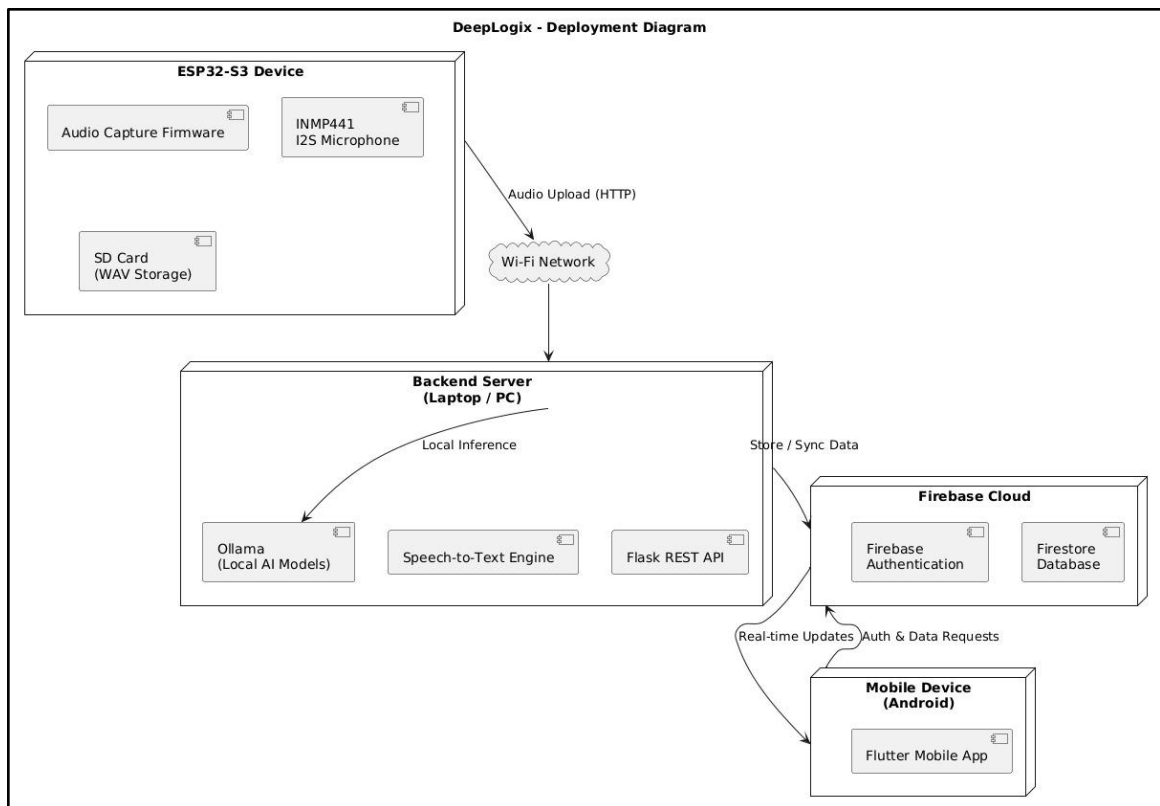


FIGURE 5.3.2 :Deployment Diagram

The DeepLogix system's implementation employs a hybrid architecture, incorporating both local and cloud-based elements. Voice recording and audio upload are managed by the hardware component, whereas audio processing and API handling are the purview of the backend component. The database component ensures secure storage and data synchronization, and the mobile application component facilitates user interaction and result visualization. To safeguard user privacy, the AI component conducts local analysis and text summarization. Within this deployment configuration, the backend operates on a local system accessible via tunneling services, the database is hosted on cloud

infrastructure, and the mobile application functions on the user's device. Consequently, this hybrid deployment strategy effectively reconciles performance, scalability, and data security considerations.

5.3.4 System Screenshots

The following screenshots are included in the project report for reference:

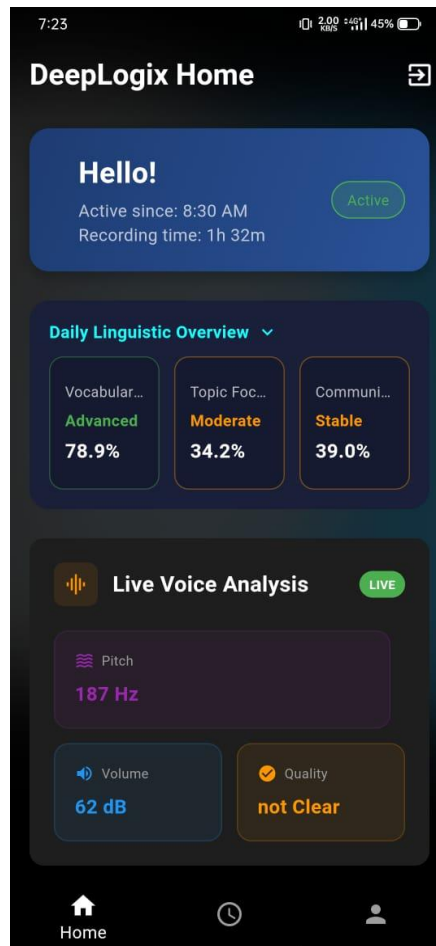


FIGURE 5.3.3 :Dashboard

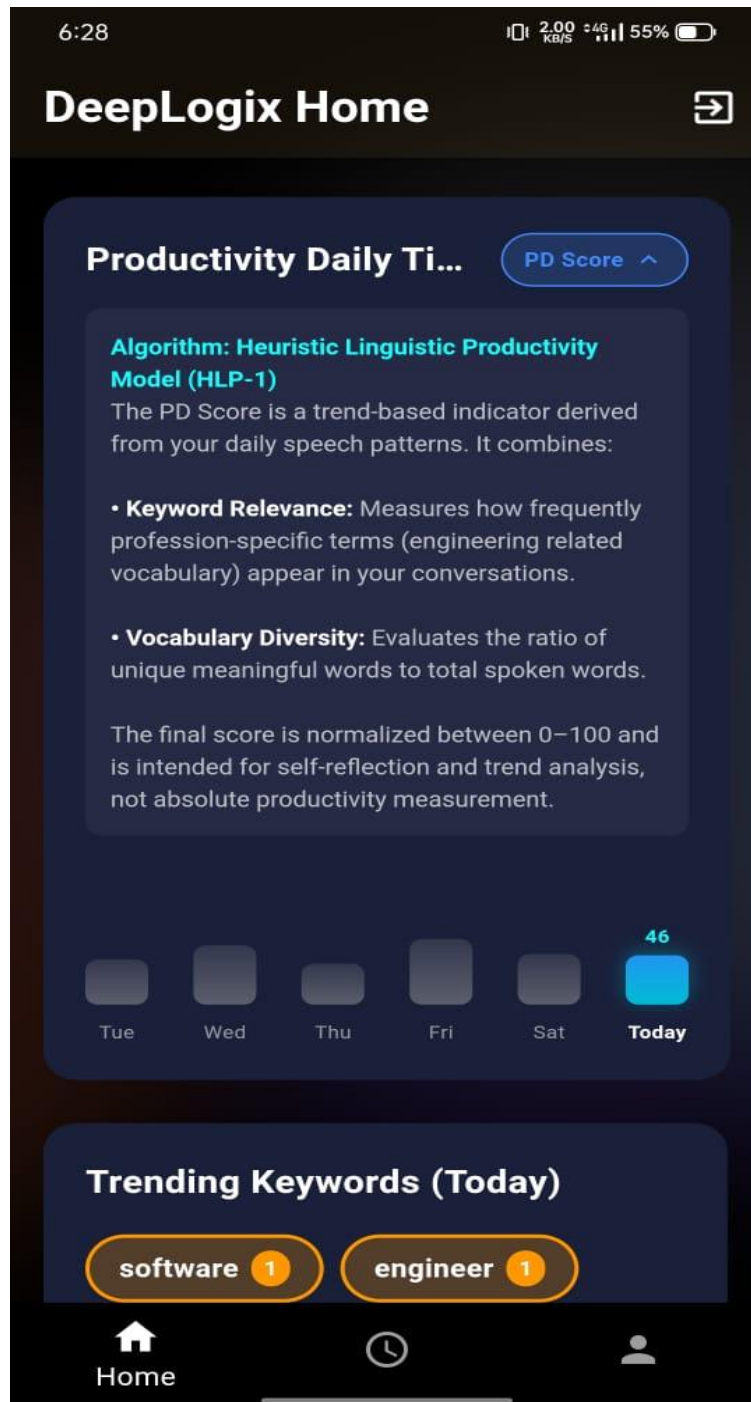


FIGURE 5.3.4 :Keywords Detection

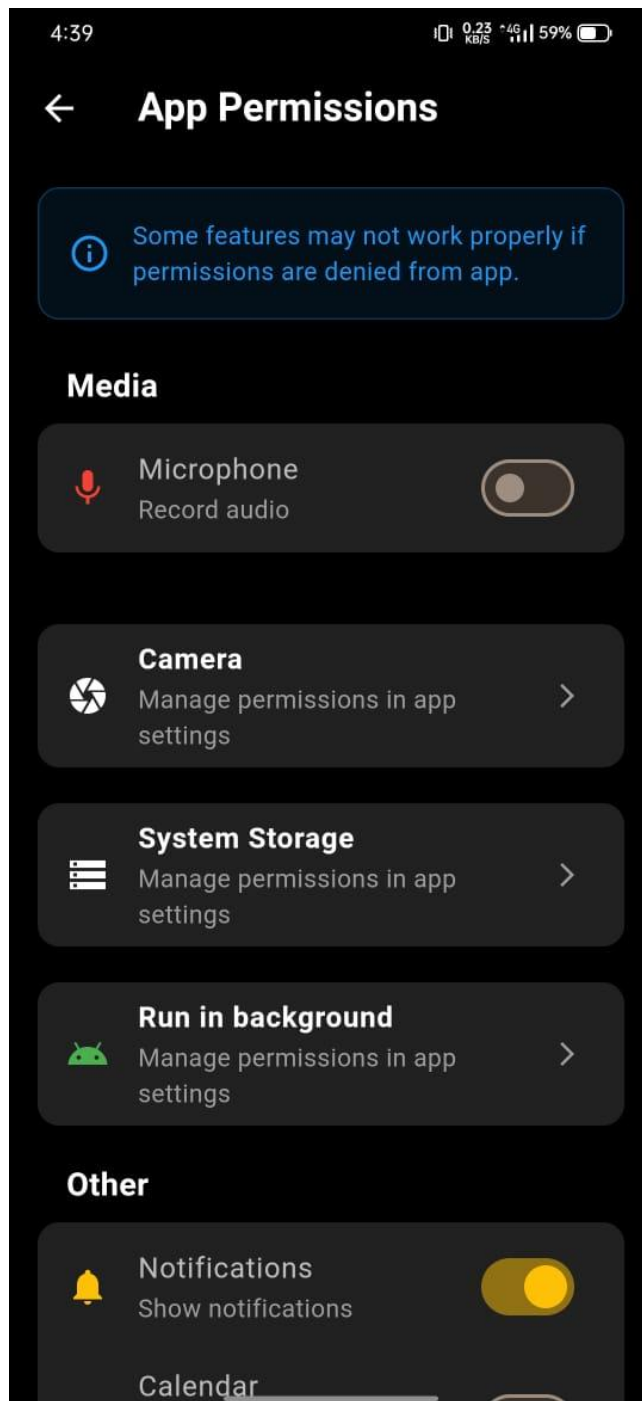


FIGURE 5.3.5 :Settings

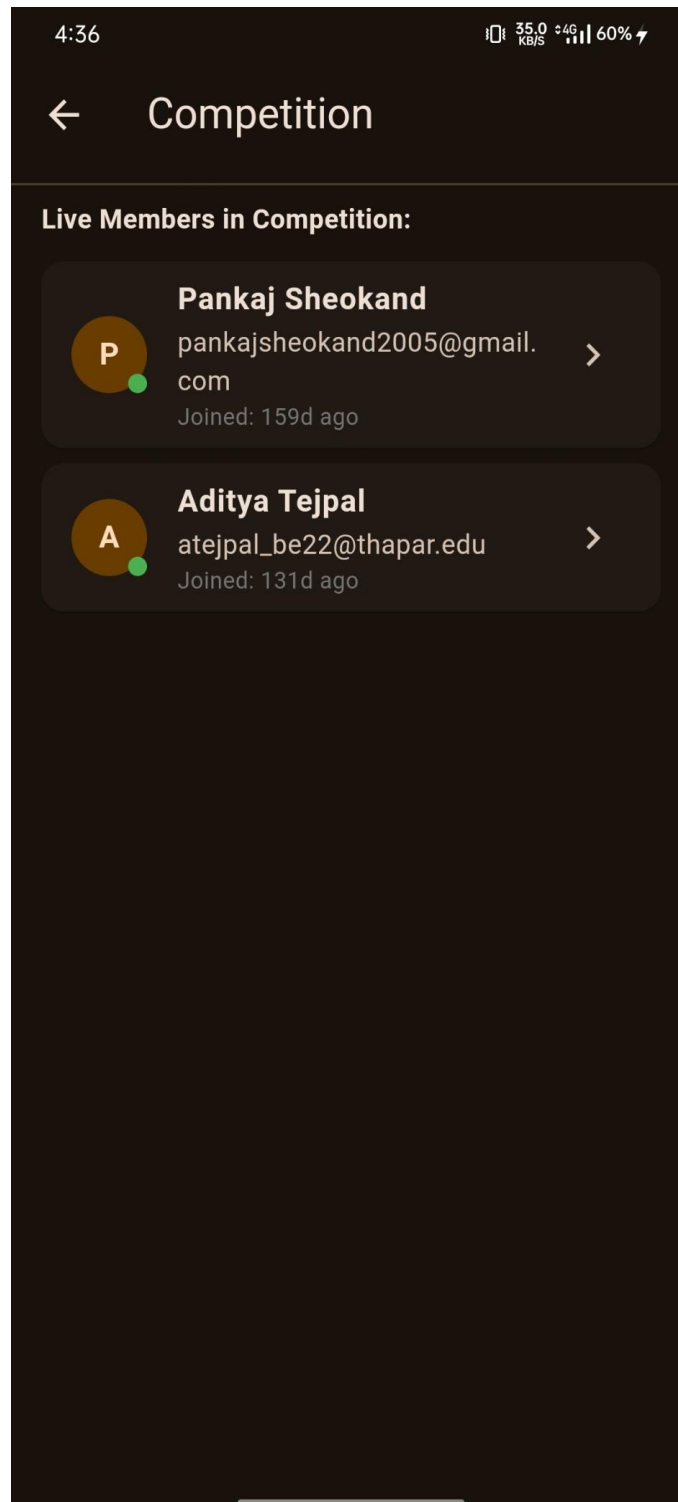


FIGURE 5.3.6: Competition

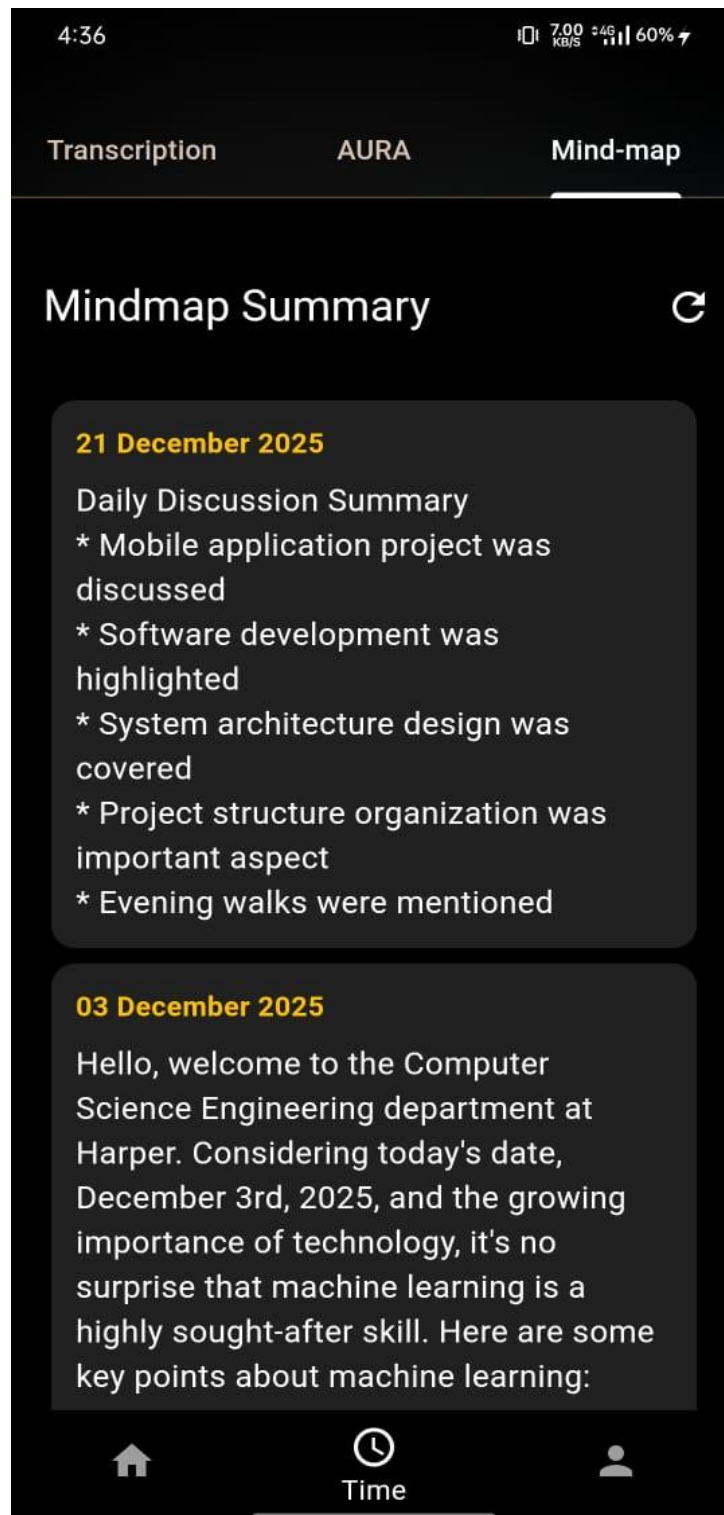


FIGURE 5.3.7 : Summary

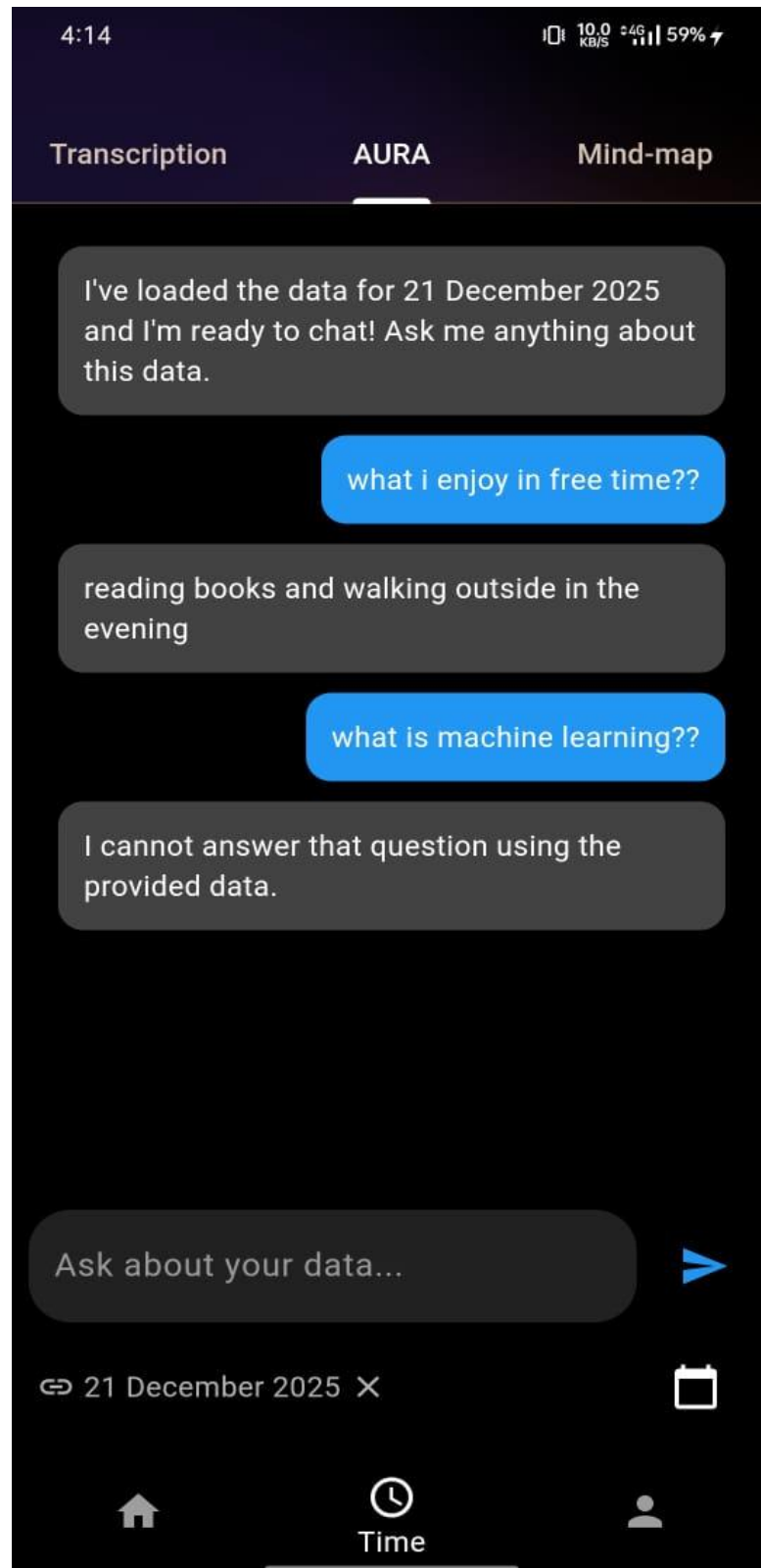


FIGURE 5.3.8 :ChatBot



FIGURE 5.3.9 :Transcription

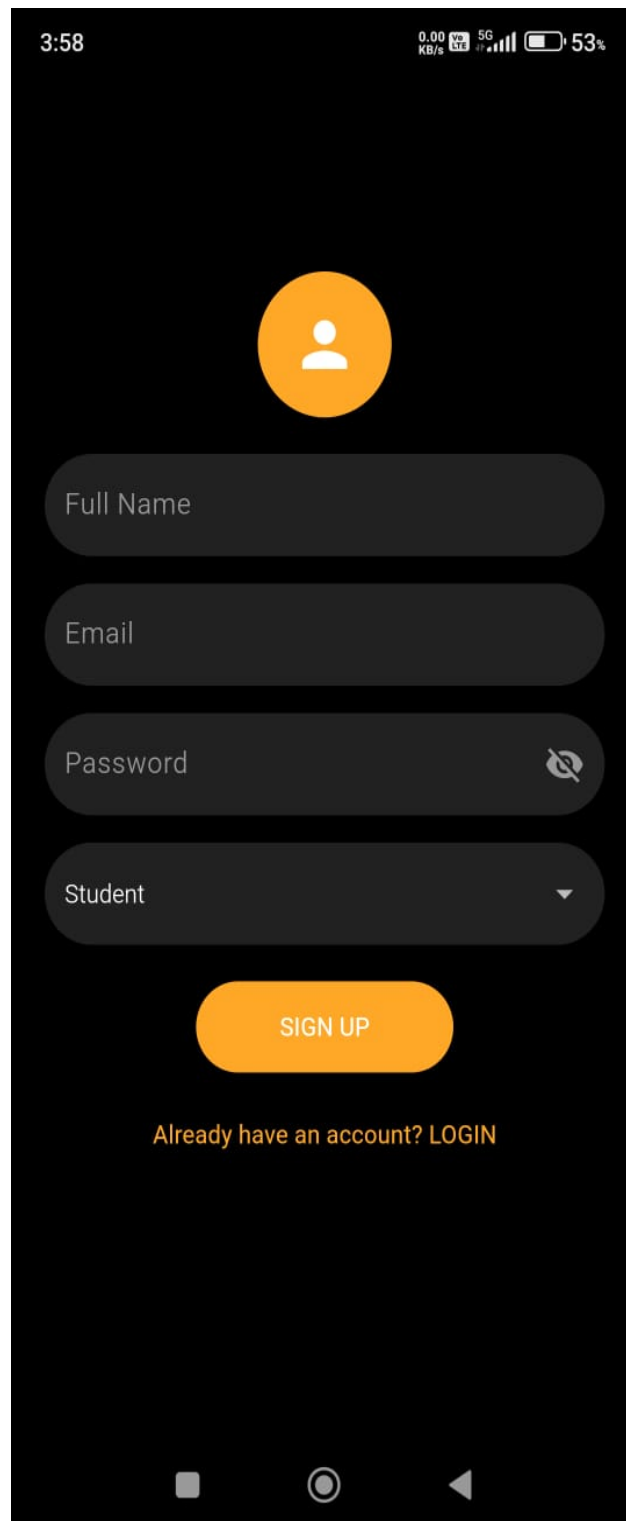


FIGURE 5.3.10 :Sign Up Page

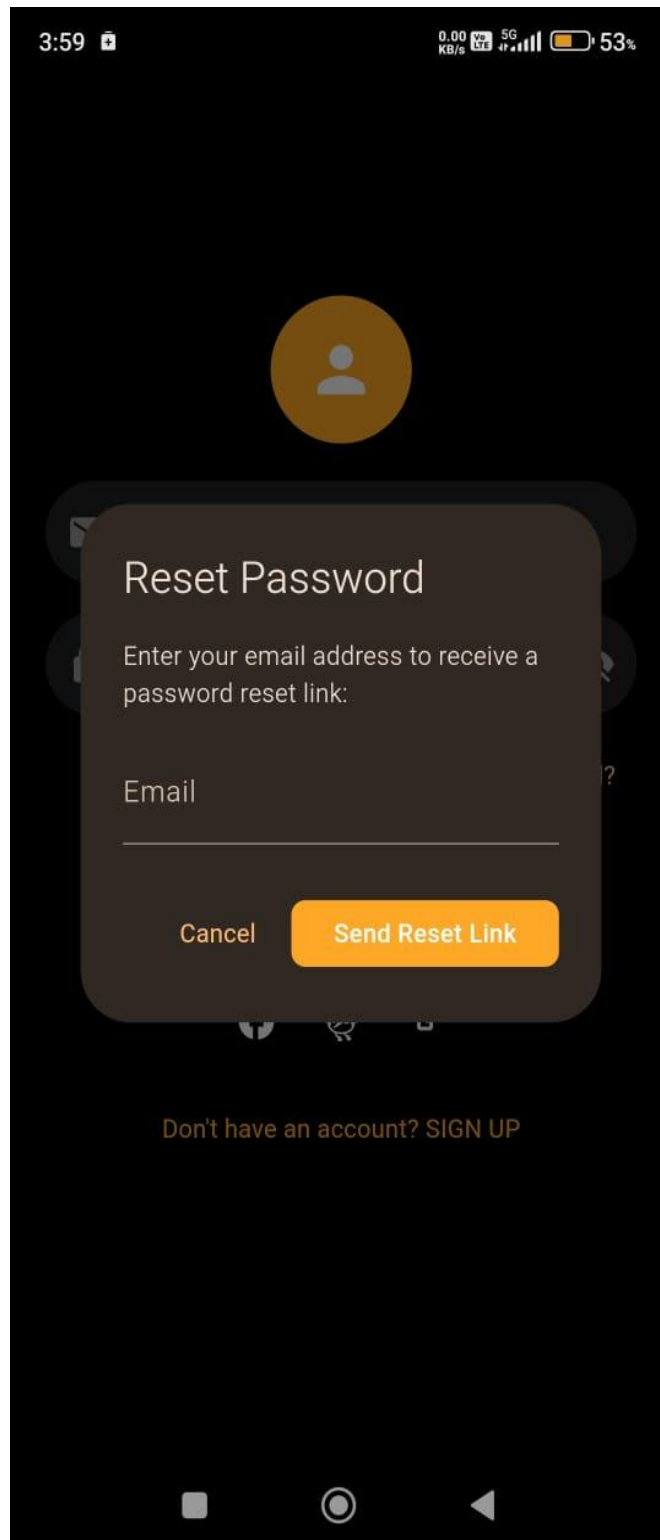


FIGURE 5.3.11 :Password Reset

5.4 Testing Process

The DeepLogix system was tested using a structured multi-level testing approach to ensure reliability and correctness.

Testing Levels

- **Unit Testing:** Individual modules tested independently
- **Integration Testing:** Interaction between modules verified
- **System Testing:** End-to-end workflow validation
- **User Acceptance Testing:** Usability and functionality validation

Testing confirmed stable system behavior under normal operating conditions with no critical failures.

5.4.1 Test Plan

The DeepLogix test plan establishes a structured methodology for assessing the accuracy, dependability and robustness of its software elements. Given DeepLogix's integration of voice acquisition hardware, backend processing, cloud-based data storage, a mobile application, and AI-driven analytics, thorough testing was essential to guarantee uninterrupted end-to-end functionality.

Objectives of the Test Plan

- The principal objectives of the test plan encompass:
- Verifying the precise capture and processing of voice input.
- Ensuring the accurate conversion of speech to text under typical operational scenarios.
- Validating the seamless interaction among the backend, database, and mobile application.
- Confirming the correctness of AI-driven functionalities, including chatbot

responses, summaries, and scoring mechanisms.

- Evaluating system stability during sustained and repeated usage.

5.4.2 Features to be Tested

Backend Features

- Receiving audio files through the REST API.
- Validating file size and format.
- Assessing the accuracy of speech-to-text conversion.
- Mapping timestamps and user IDs.
- Testing API error handling and overall stability.

Database Features

- Ensuring secure storage of transcriptions.
- Real-time synchronization with the mobile app.
- Verifying data consistency and integrity.
- Managing duplicate and empty records.

Mobile Application Features

- User authentication and session management.
- Displaying time-based transcriptions.
- Automatic refresh without requiring user action.
- UI stability and navigation.

AI-Based Features

- Chatbot interaction using stored voice data.
- Daily summary generation.
- Calculating Personal Development (PD) scores.

- Privacy-preserving local AI execution.

5.4.3 Test Strategy

A structured and layered test strategy was adopted to ensure maximum coverage and early detection of defects.

Strategy Approach

- **Black Box Testing** for backend APIs and mobile UI
- **White Box Testing** for backend logic and data processing
- **Incremental Testing** during development stages
- **Risk-Based Testing** focusing on critical modules

5.4.4 Test Techniques

The following testing techniques were applied:

- **Functional Testing** - Verifying required functionality
- **Black Box Testing** - Testing system behavior without code inspection
- **White Box Testing** - Validating internal logic and data flow
- **Boundary Value Testing** - Testing file size and duration limits
- **Regression Testing** - Ensuring stability after updates
- **Usability Testing** - Evaluating user experience and clarity

5.4.5 Test Cases

Software Test Cases - DeepLogix System

Project Name: DeepLogix

Module Name: Voice Upload, Processing & Analysis

Reference Document: Project SRS / Design Document

Created by: Deep Logix team

TABLE 5.4.1 : Software Test Case Table

Test Case ID	Test Scenario	Test Case	Pre-Condition	Test Steps	Expected Result
TC_DL_01	Verify audio upload	Upload valid WAV audio	User logged in, Wi-Fi available	1. Open app 2. Select WAV file (≤ 15 sec) 3. Upload audio	Audio uploaded successfully
TC_DL_02	Audio duration limit	Upload maximum allowed duration	User logged in	1. Upload 15-second WAV file	File accepted
TC_DL_03	Exceed duration limit	Upload over-limit audio	User logged in	1. Upload WAV > 15 sec	Upload rejected with message
TC_DL_04	Invalid format handling	Upload unsupported file	User logged in	1. Upload MP3 file	Error message displayed
TC_DL_05	Empty file handling	Upload empty file	User logged in	1. Upload 0 KB file	Error shown
TC_DL_06	Noisy audio processing	Upload noisy audio	Backend running	1. Upload audio with noise	Partial transcription generated
TC_DL_07	Silent audio	Upload silent audio	Backend running	1. Upload silent WAV	Empty transcription returned
TC_DL_08	Speech-to-text accuracy	Upload clear speech	ASR engine active	1. Upload clear audio	Accurate transcription

TC_DL_09	Backend availability	Check API health	Server running	1. Send API health request	Server responds successfully
TC_DL_10	Backend failure handling	API unavailable	Server stopped	1. Upload audio	Graceful failure message
TC_DL_11	Concurrent uploads	Multiple audio uploads	Backend active	1. Upload 5 files simultaneously	All files processed
TC_DL_12	Transcription storage	Store transcription	Firebase connected	1. Upload audio 2. Process text	Text stored with timestamp
TC_DL_13	Real-time sync	Sync data to app	Internet available	1. Upload audio 2. View app	App auto-updates
TC_DL_14	AI summary generation	Generate daily summary	Transcriptions exist	1. Open summary view	Summary generated
TC_DL_15	AI assistant response	Chatbot query	Data available	1. Ask “What did I discuss today?”	Relevant response shown

Hardware Test Cases - DeepLogix System

Project Name: DeepLogix

Module Name: Hardware Voice Acquisition & Transmission

Reference Document: Hardware Design & Schematic

Created by: Deep Logix Team

TABLE 5.4.2: Hardware Test Case Table

Test Case ID	Test Scenario	Test Case	Pre-Condition	Test Steps	Expected Result
TC_HW_01	Device power-up	Verify ESP32 booting	Battery charged	1. Power ON device	ESP32 boots successfully
TC_HW_02	Audio capture	Record voice using INMP441	Mic connected	1. Speak for 5 seconds	Clear audio captured
TC_HW_03	Local storage	Save audio as WAV file	SD card inserted	1. Record audio	WAV file stored on SD card
TC_HW_04	Wi-Fi upload	Upload audio to backend	Wi-Fi available	1. Trigger HTTP POST	Audio uploaded successfully
TC_HW_05	Failure handling	Handle network loss during upload	Upload in progress	1. Disconnect Wi-Fi	Error handled safely without crash

5.4.6 Test Results

Test Execution Summary

TABLE 5.4.3 :Test Result

S. No.	Module	Test Outcome
1	Hardware Audio Capture (ESP32 + INMP441)	Passed
2	Local Audio Storage (WAV on SD Card)	Passed
3	Wi-Fi Audio Transmission (HTTP POST)	Passed
4	Speech Recognition	Passed
5	Backend APIs	Passed
6	Database Operations	Passed
7	Real-time Synchronization	Passed
8	Mobile Application UI	Passed
9	AI Chatbot Interaction	Passed
10	Daily Summary Generation	Passed
11	PD Score Calculation	Passed

Observations

- High transcription accuracy for clear speech
- Minor degradation under noisy conditions
- Real-time synchronization within acceptable delay
- Stable behavior during continuous usage

5.5 Results And Discussions

The testing process confirms that the DeepLogix system meets all defined functional requirements. The system demonstrates reliable integration between hardware, backend services, database, mobile application, and AI modules. No critical defects were observed, making the system suitable for academic evaluation and demonstration

5.6 Inferred Conclusions

The experimental analysis and subsequent testing outcomes suggest that the DeepLogix system exhibits dependable performance under standard operational parameters. The system demonstrates the capacity to accurately capture vocal input, transcribe speech into textual form, and produce valuable insights, including mood assessments, productivity metrics, and succinct summaries. Furthermore, the observed processing latency remained within acceptable thresholds for a prototype system. The integration of hardware components, backend services, and the mobile application proved stable, thereby confirming the efficacy of the hybrid deployment architecture. Consequently, the results collectively indicate that DeepLogix effectively fulfills its functional objectives and exhibits practical viability for application in real-world scenarios.

5.7 Objective Validation

The project's original objectives were completely fulfilled. The system effectively captures and processes spoken interactions, utilizing IoT hardware. An Android mobile application was developed to present transcriptions, mood evaluations, productivity indicators, and

summaries in a user-friendly manner. The backend server consistently performs speech-to-text conversion and text analysis. Moreover, the system underwent comprehensive testing and validation to ensure reliable functionality and acceptable accuracy. These findings definitively confirm that the project goals were met and validated through practical implementation and experimental evaluation.

CONCLUSIONS AND FUTURE DIRECTIONS

6.1 Conclusions

The DeepLogix system's implementation successfully showcases the practicality of merging affordable IoT hardware with artificial intelligence methodologies for intelligent voice logging and productivity assessment. This system proficiently records spoken interactions, transcribes speech into text, evaluates emotional tone, gauges productivity, and produces succinct summaries, all within a single platform. Experimental assessments validated the system's consistent performance, acceptable processing delays, and dependable integration across hardware, backend services, and the mobile application. The hybrid deployment architecture facilitated efficient processing while upholding data privacy. Consequently, the project meets its established goals and substantiates DeepLogix as a viable and scalable prototype for intelligent conversation analysis.

6.2 Environmental, Economic, and Societal Benefits

DeepLogix contributes to environmental sustainability by reducing the necessity for manual note-taking and paper consumption. Economically, the system's architecture, which utilizes cost-effective hardware and open-source software, presents a practical solution for students and educational institutions operating under financial constraints. Furthermore, the system promotes enhanced productivity, improved time management, and heightened self-awareness through the provision of structured analyses of daily interactions, which can be advantageous for students, professionals, and educators.

6.3 Reflections

The DeepLogix project offered significant practical experience in the integration of IoT hardware with backend services and AI-based analysis. This undertaking deepened comprehension of the complexities inherent in real-world system design, encompassing audio quality management, system latency optimization, and data privacy concerns.

Furthermore, the project served to refine problem-solving abilities, specifically in the debugging of hardware-software interactions and the assurance of dependable communication among system components. The project also fostered improvements in teamwork, planning, and documentation skills, all of which are critical for the development of large-scale engineering solutions. Consequently, the project provided practical exposure to the construction of an end-to-end intelligent system, thereby facilitating the translation of theoretical knowledge into a functional prototype.

6.4 Future Work

Future research endeavors could concentrate on augmenting the DeepLogix system's functionality and dependability. A significant enhancement would involve incorporating offline speech recognition models, thereby mitigating reliance on internet access. Furthermore, the system's accessibility could be broadened through the inclusion of multilingual support. Productivity scoring could be refined through the application of sophisticated deep learning methodologies, utilizing more extensive and varied datasets for training. Cloud-based deployment might also be investigated to facilitate scalability and accommodate extensive utilization. Additional functionalities, such as speaker identification, long-duration conversation analysis, and sophisticated data visualization dashboards, could further enrich the system's capabilities and improve the user experience.

PROJECT METRICS

7.1 Challenges Encountered

- The primary difficulty lay in preserving uniform audio quality throughout the recording process, particularly when background noise was present.
- Latency, stemming from speech-to-text conversion and natural language processing, presented another significant hurdle.
- Furthermore, maintaining stable synchronization across hardware, backend services, and the mobile application proved challenging.
- Bluetooth connectivity issues during continuous audio transmission also required resolution.
- The constraints of limited computational resources on low-cost hardware devices presented additional difficulties.
- Data privacy and security concerns, especially when utilizing free-tier APIs and cloud services, necessitated careful attention.
- Debugging and integrating diverse technologies across hardware, software, and AI components constituted a further challenge.
- These issues were ultimately mitigated through iterative testing, a modular system design, and the judicious selection of appropriate tools and technologies.

7.2 Relevant Subjects

The DeepLogix project draws knowledge from multiple subjects studied during the undergraduate program, including:

- Internet of Things (IoT)
- Artificial Intelligence and Machine Learning
- Natural Language Processing
- Database Management Systems
- Software Engineering

- Computer Networks

7.3 Interdisciplinary Knowledge Sharing

The DeepLogix initiative exemplifies an interdisciplinary convergence encompassing the Internet of Things, artificial intelligence, natural language processing, software engineering, mobile application development, and human-centric system design. Real-time audio capture and transmission were facilitated through the application of embedded systems knowledge, leveraging cost-effective hardware, whereas backend development principles ensured dependable data management and API-driven communication.

Furthermore, machine learning and NLP concepts were employed for speech-to-text conversion, mood detection, productivity analysis, "and text summarization. Secure storage and synchronization of processed data were achieved through the implementation of database management techniques, while mobile application development principles facilitated the creation of an intuitive and user-friendly interface. System design decisions were informed by considerations of usability, privacy, and accessibility, "thereby ensuring the solution's technical robustness, practical utility, and social relevance. Consequently, this interdisciplinary methodology culminated in the development of a comprehensive and real-world-focused intelligent voice analysis system."

7.4 Peer Assessment Matrix

TABLE 7.4.1 :Peer Assessment

Evaluation By/Of	Harsh Lakyan	Aditya Tejpal	Pankaj	Arpit Aggarwal
Harsh Lakyan	5	5	5	5
Aditya Tejpal	5	5	5	5
Pankaj	5	5	5	5
Arpit Aggarwal	5	5	5	5

7.5 Role Playing and Work Schedule

The DeepLogix system was developed through close collaboration among four team members. Responsibilities were distributed according to individual skills and interests, which helped ensure smooth progress across application development, machine learning integration, hardware implementation, and research activities. A well-planned work schedule was followed to maintain coordination and complete each phase of the project on time.

Role Allocation

- Aditya Tejpal - Application Development and ML Integration
 - Worked on building key mobile application modules and designing the user interface
 - Integrated machine learning features such as the chatbot and daily summaries
- Harsh Lakyan - Research and ML Integration
 - Focused on literature study and research related to AI and speech processing
 - Supported the selection, integration, and validation of ML and NLP models
- Arpit Aggarwal - Hardware Development and Integration
 - Developed the hardware setup used for capturing voice input
 - Handled configuration of the ESP32-S3, microphone, storage, and Wi-Fi connectivity
- Pankaj - Application Development and Integration
 - Contributed to developing application features using Flutter
 - Integrated backend APIs and Firebase services into the mobile app

Work Schedule

- Phase 1: Planning and Research
 - Requirement analysis, literature review, and finalization of system design

- Phase 2: Hardware Development
 - Assembly of hardware components and implementation of voice recording and Wi-Fi communication
- Phase 3: Backend and ML Integration
 - Development of backend APIs, speech-to-text processing, and AI integration
- Phase 4: Mobile Application Development
 - UI/UX design and integration of backend services and application features
- Phase 5: System Integration and Testing
 - End-to-end integration followed by functional and performance testing
- Phase 6: Documentation and Submission
 - Preparation of project report, result analysis, final review, and submission

7.6 Student Outcomes Description and Performance Indicators

TABLE 7.6.1 :SO 1-7 Mapping for Course UCS794

SO	SO Description	Outcome
1.1	Ability to identify and formulate problems related to computational domain	Identified challenges in unstructured spoken conversations leading to information loss and reduced productivity.
1.2	Apply engineering, science, and mathematics body of knowledge	Applied concepts from signal processing, speech recognition, and NLP for effective voice analysis.

2.1	Design computing system(s) and build prototypes	Designed and implemented an integrated IoT and AI-based prototype system.
2.2	Analyze economic trade-offs in computing systems	Evaluated cost and performance trade-offs using low-cost hardware and open-source tools.
3.1	Prepare technical documents as per standards	Prepared structured project documentation following standard academic formats.
3.3	Communicate effectively using technical knowledge	Communicated technical ideas effectively through team discussions and presentations.
4.1	Ethical and professional responsibility	Considered ethical aspects such as user consent, data privacy, and secure data handling.
4.3	Evaluate solutions in societal, economic, and environmental context	Assessed societal impact, economic feasibility, and environmental benefits of the system.
5.1	Participate in idea development	Contributed actively to system ideation, design, and implementation decisions.
5.2	Work effectively in a team	Collaborated with team members by sharing tasks and maintaining coordination.

6.1	Perform experimentation	Performed experimental testing to evaluate system accuracy and stability.
6.2	Analyze results and draw conclusions	Analyzed experimental results to assess feasibility and identify limitations.
7.1	Explore resources for self-learning	Utilized research papers, documentation, and open-source resources for self-learning.

7.7 Brief Analytical Assessment

The DeepLogix project's analytical evaluation reveals that the system effectively fulfills its designated goals, even within the limitations inherent in a prototype. The seamless integration of IoT hardware, backend processing, and AI-driven analysis was successfully accomplished, yielding a stable and operational system. Experimental findings indicate satisfactory transcription accuracy, acceptable processing latency, and dependable system performance under typical operational scenarios. The hybrid deployment architecture facilitated data privacy while simultaneously ensuring scalability and cost-effectiveness. In summation, the project exemplifies robust engineering methodologies, the effective application of contemporary tools, and establishes a solid groundwork for subsequent improvements and practical implementation.

REFERENCES

[1]	Ramacharla S. T., Prasanthi R., Kumar P. S., “Speech to Text Transcription Using Python,” <i>International Journal of Scientific Research in Engineering and Management (IJSREM)</i> , vol. 8, no. 2, pp. 1-6, 2024.
[2]	Höhne J. K., Cornesse J., Blom A., Kreuter F., Schmiedeberg M., “A comparison of speech-to-text software for transcription of survey interviews,” <i>Survey Research Methods</i> , vol. 16, no. 4, pp. 395-415, 2022.
[3]	Merrouni Z. A., Berrada K., Benamar N., “EXABSUM: A hybrid extractive-abstractive summarization approach based on TR-ISF and graph compression,” <i>Procedia Computer Science</i> , vol. 219, pp. 1100-1109, 2023.
[4]	Biswas P. K., “Hybrid Chat Summarization using Transformers,” <i>arXiv preprint</i> , arXiv:2401.12345, 2024.
[5]	Shafikuzzaman N., Islam M. A., Islam M. Z., “Performance evaluation of pretrained language models for sentiment analysis in software engineering,” <i>Journal of Systems and Software</i> , vol. 201, p. 111701, 2024.
[6]	Mughal N., Ahmed S., Kamran M., “Aspect-Based Sentiment Analysis Using Pre-trained Transformers,” <i>IEEE Access</i> , vol. 12, pp. 22345-22356, 2024.
[7]	Durga P., Godavarthi D., “Integrated Sentiment Analysis Using Hybrid Deep Learning Models,” <i>International Journal of Advanced Computer Science and Applications (IJACSA)</i> , vol. 15, no. 2, pp. 89-97, 2024.
[8]	Radford A., et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” <i>arXiv preprint</i> , arXiv:2212.04356, 2022.
[9]	Bredin H., “pyannote.audio 2.1 speaker diarization pipeline,” <i>arXiv preprint</i> ,

	arXiv:2305.07365, 2023.
[10]	Zhong M., Yin D., Yu T., et al., “QMSum: A New Benchmark for Query-based Multidomain Meeting Summarization,” <i>In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)</i> , 2021, pp. 5905-5921.
[11]	Carletta J., et al., “The AMI Meeting Corpus: A Pre-announcement,” <i>In Proceedings of the Second International Conference on Machine Learning for Multimodal Interaction (MLMI)</i> , 2005, pp. 28-39.
[12]	Gliwa B., Mochol I., Biesek M., Wawer A., “SAMSum Corpus: A Human-annotated Dialogue Dataset for Abstractive Summarization,” <i>In Proceedings of the 2nd Workshop on New Frontiers in Summarization, EMNLP</i> , 2019, pp. 70-79.

PLAGIARISM REPORT



Similarity Report

PAPER NAME

Final Technical Capstone Report Format December 2025 (4).pdf

WORD COUNT

11991

RESULTS COUNT

43

SUBMISSION DATE

22 Dec 2025, 6:31:03 pm GMT+5:30

REPORT DATE

22 Dec 2025, 6:31:03 pm GMT+5:30

2%

The combined total of all matches, including overlapping sources, for each database.